Instituto Superior
de Engenharia
Politécnico de Coimbra

DEPARTAMENTO DE INFORMÁTICA E SISTEMAS

WAZUH

Relatório de Licenciatura em Engenharia Informática

Autores

José Pedro Leal Neto - 2023140616

Renato Augusto Carvalho Santos - 2019143713

Orlando Nogueira de Souza dos Santos - 2023134097

Unidade Curricular

SEGURANÇA 25/26

Professor

Luís Eduardo Faria dos Santos

Coimbra, 08 Junho de 2026

INSTITUTO POLITÉCNICO
DE COIMBRA

INSTITUTO SUPERIOR
DE ENGENHARIA
DE COIMBRA

Índice

Índice i
Índice de tabelas ii
Índice de figuras iii
Lista de abreviaturas v
1 Introdução 1
2 Proposta 2
3 Topologia 4
4 Ferramenta Wazuh 5
5 Comunicação entre Componentes 14
6 Setup e Configuração da Topologia 18
7 Sistemas IDS e IPS 19
8 Sistemas XDR e HIDS 20
9 Aplicação de Políticas de Segurança 21
10 Análise das Experiências Realizadas 33
11 Conclusão 54
Referências bibliográficas 56
Anexos 59

Índice de tabelas

1 Endereçamento da topologia 4

2 Portos principais utilizados na comunicação da infraestrutura Wazuh analisada 17

Índice de figuras

1 Wazuh logo 2

2 Topologia GNS3 4

3 Exemplo de Componentes Funcionamento Wazuh 5
4 Wazuh Manager Architecture 6
5 Arquitetura Indexer 7
6 Wazuh Dashboard Inteface 8
7 Wazuh Agent Architecture 9
8 Arquitetura Wazuh 10
9 Arquitetura All-in-one 11
10 Arquitetura Single-Node 12
11 Arquitetura Multi-Node 13

12 Lista de reputação presente no conjunto de regras do Wazuh 21
13 Lista de IPs maliciosos configurada no Wazuh 22

14 Pedido HTTP e subsequente tentativa de conectividade que falha 34
15 Ligação HTTP com pedido e resposta efetuados 36
16 Ligação TCP entre servidor Wazuh e agente 36
17 Pacotes ICMP echo request sem resposta 36
18 Teste de força bruta e subsequente tentativa de conectividade que falha . 37
19 Estabelecimento de uma das sessões SSHv2 41
20 Segmentos TCP que validam o sistema IPS 41
21 Encerramento das ligações TCP por parte do atacante e teste de conectividade 41
22 *Port scan* usando *nmap* 42
23 Pacotes TCP do *nmap* e avisos ICMP *filtered* 46
24 Primeiro pacote Syslog e ligação Telnet enviado 46
25 Pacotes Syslog provenientes do router e teste do *ping* no atacante 46
26 Alguns pacotes da segunda ligação Telnet e o pacote Syslog final da experiência 47
27 Ataque ao agente Xubuntu e posterior teste de conectividade que falha no atacante 47
28 Pacotes filtrados e pacotes ICMP sem resposta 48

29 Ataque *SYN flood* efetuado no atacante Kali-Linux 49
30 Pedidos de ligação fraudulentos e tentativas de resposta do router 52
31 Início da ligação Telnet 52
32 Router a recusar pacotes maliciosos 53

Lista de abreviaturas

ACL	*Access List*
ACK	*Acknowledgment*
AES	*Advanced Encryption Standard*
API	*Application Programming Interface*
CA	*Certification Authority*
CBC	*Cipher Block Chaining*
CDB	*Constant Database*
CIA	*Confidencialidade, Integridade e Autenticidade*
CPU	*Central Processing Unit*
DHCP	*Dynamic Host Configuration Protocol*
DNS	*Domain Name System*
FIN	*Finish*
FTP	*File Transfer Protocol*
GNS3	*Graphical Network Simulator-3*
HIDS	*Host-based Intrusion Detection System*
HIPS	*Host-based Intrusion Prevention System*
HTML	*Hypertext Markup Language*
HTTP	*Hypertext Transfer Protocol*
HTTPS	*Hypertext Transfer Protocol Secure*
ICMP	*Internet Control Message Protocol*
IDS	*Intrusion Detection System*
IEEE	*Institute of Electrical and Electronics Engineers*
IOC	*Indicator of Compromise*
IOS	*Internetwork Operating System*
IP	*Internet Protocol*
IPS	*Intrusion Prevention System*
ISEC	*Instituto Superior de Engenharia de Coimbra*
JSON	*JavaScript Object Notation*
MITM	*Man in the Middle*

NAT	*Network Address Translation*
OSI	*Open Systems Interconnection*
OVA	*Open Virtual Appliance*
PAM	*Pluggable Authentication Modules*
QEMU	*Quick Emulator*
RAM	*Random Access Memory*
RST	*Reset*
SSH	*Secure Shell*
SRV	*Server*
SSL	*Secure Sockets Layer*
SYN	*Synchronization Sequence Number*
TCP	*Transmission Control Protocol*
TLS	*Transport Layer Security*
UDP	*User Datagram Protocol*
VM	*Virtual Machine*
VTY	*Virtual Teletype*
XDR	*Extended Detection and Response*
XML	*Extensible Markup Language*

1 Introdução

Como parte de um projeto para a unidade curricular de Segurança, este relatório tem como objetivo estudar a ferramenta Wazuh, as suas funcionalidades principais, a sua arquitetura e a sua aplicação em cenários de deteção e prevenção de intrusões. Este documento é também considerado um bom guia para iniciantes, com uma pequena documentação e guia prático em anexos de como proceder a montagem da topologia e testes da ferramenta, toda a pesquisa realizada esta presente na bibliografia.

A implementação de um ambiente laboratorial virtual é essencial para a realização de testes e experiências controladas. Desta forma, torna-se possível observar o comportamento dos sistemas de deteção de intrusão (IDS) e dos sistemas de prevenção de intrusão (IPS), bem como compreender as diferenças entre ambos.

O Wazuh é uma ferramenta bastante completa em termos de IDS, sendo limitada em alguns requisitos de IPS, mas em todo é super aconselhável para por exemplo a monitorização de diversas máquinas virtuais, servidores DNS[1], FTP[2] e dentre outros, reportando em tempo real os acontecimentos via syslog ou associação de agentes.

O Wazuh possui várias formas de implementação sendo elas All-in-one, Single-node e Multi-node. Neste projeto será utilizada uma abordagem All-in-one, na qual o Wazuh Manager, o Wazuh Indexer e o Wazuh Dashboard se encontram concentrados na mesma máquina. Esta escolha permite simplificar a configuração do laboratório e focar a análise nos testes de segurança realizados, sendo a ideal para ambientes de testes laboratoriais.

Para ser comprovada a eficácia da ferramenta com sistemas IPS e IDS, foram utilizados métodos de *paintesting* de forma a demonstrar o potencial do sistema, ataques como scan de portos, syn-floods, brute-force e associação de uma máquina a uma lista de ips maliciosos foram realizados.Todas estas experiências foram realizadas apenas em contexto laboratorial, não sendo utilizadas para efeitos ilegais.

A análise de todas as experiências encontra-se documentada, principalmente dos comportamentos como a comunicação entre dispositivos e troca de pacotes analisada, também a configuração e definição das regras e comportamentos perante a ameaças informáticas.

2 Proposta

2.1 Objetivo da Proposta

A proposta deste trabalho consiste na criação de um laboratório virtual para estudar a utilização do Wazuh[3] em cenários de monitorização, deteção e prevenção de intrusões. O objetivo principal é compreender de que forma a ferramenta recolhe eventos, gera alertas e permite analisar comportamentos suspeitos numa rede controlada.

O Wazuh é uma ferramenta utilizada para monitorização e deteção de segurança em hosts (HIDS/XDR), recolhendo e correlacionando eventos como logs, integridade de ficheiros, inventário, vulnerabilidades e auditorias de configuração.

Com base em regras e políticas, o Wazuh consegue detetar atividades suspeitas e gerar alertas. Estes alertas podem ser utilizados para estudar o funcionamento de sistemas IDS e apoiar a implementação de mecanismos associados a sistemas IPS.



Figura 1: Wazuh logo

2.2 Software Utilizado

Foram selecionadas algumas ferramentas que permitem criar e configurar a topologia de rede, de forma a analisar o comportamento do sistema e relatar a experiência realizada.

2.2.1 Aplicações

Para virtualização foi selecionado o VMWARE[4], para emulação da topologia foi escolhido o GNS3[5] e para análise e leitura do tráfego gerado pela rede foi escolhido o Wireshark[6].

2.2.2 Sistemas Operativos

Os sistemas operativos selecionados foram o Linux Xubuntu[7] que representará o agente monitorizado, o Amazon Linux 2023[8] que corre o Sistema Wazuh previamente configurado e o Kali Linux para realização de *paintesting*.

2.2.3 Linguagens de Programação e Marcação

As linguagens utilizadas são Python e XML[9]. O Python foi utilizado para a elaboração de um script de automação, enquanto o XML é a linguagem padrão de configuração da ferramenta Wazuh[10].

2.2.4 Ferramentas Kali

As ferramentas utilizadas para *paintesting* foram a Hydra [11], para realização de ataques de força bruta, o Nmap [12] para varrimento de portos, com apoio do guia de referência da ferramenta [13], o Hping3 [14] para geração de tráfego TCP/SYN e o Ping [15] para testes de conectividade. Para mais informação sobre as ferramentas e sua utilização pode ser consultado o anexo **J.1**.

3 Topologia

A topologia(**Figura 2**) foi organizada em duas redes principais, permitindo separar uma rede protegida de uma rede totalmente desprotegida, esta divisão facilita a análise do tráfego e a identificação dos eventos gerados durante as experiências, bem como a diferença de uma rede segura, face a uma rede totalmente fragilizada.

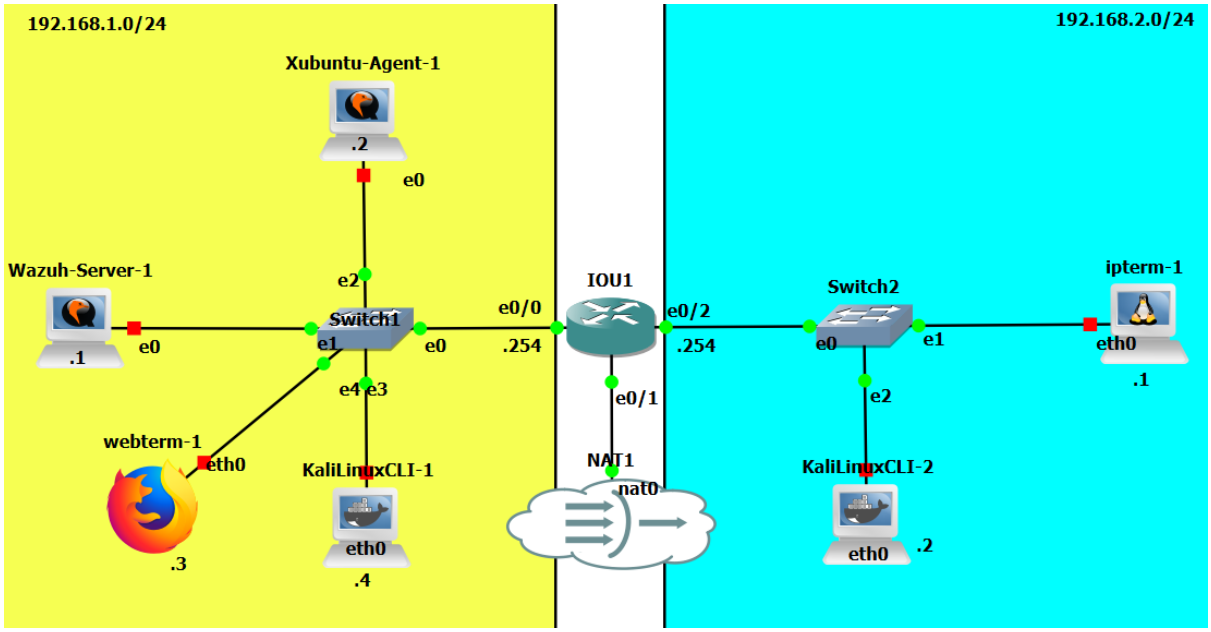


Figura 2: Topologia GNS3

3.1 Endereçamento

Designação	Rede	Máscara	Endereço IP
Wazuh-Server-1	192.168.1.0	/24	192.168.1.1
Xubuntu-Agent-1	192.168.1.0	/24	192.168.1.2
Webterm-1	192.168.1.0	/24	192.168.1.3
KaliLinuxCLI-1	192.168.1.0	/24	192.168.1.4
ipterm-1	192.168.2.0	/24	192.168.2.1
KaliLinuxCLI-2	192.168.2.0	/24	192.168.2.2

Tabela 1: Endereçamento da topologia

4 Ferramenta Wazuh

O Wazuh [3] possui uma vasta gama de funcionalidades e pode ser implementado através de diferentes arquiteturas. Este capítulo apresenta os seus principais componentes, a forma como estes comunicam entre si e as arquiteturas mais comuns. Esta contextualização é importante para compreender a configuração utilizada no laboratório e a forma como os eventos de segurança são recolhidos e analisados.

4.1 Componentes

O funcionamento do Wazuh depende da interação entre vários componentes. Cada um tem uma função específica no processo de recolha, indexação, visualização e análise dos eventos (**Figura 3**).

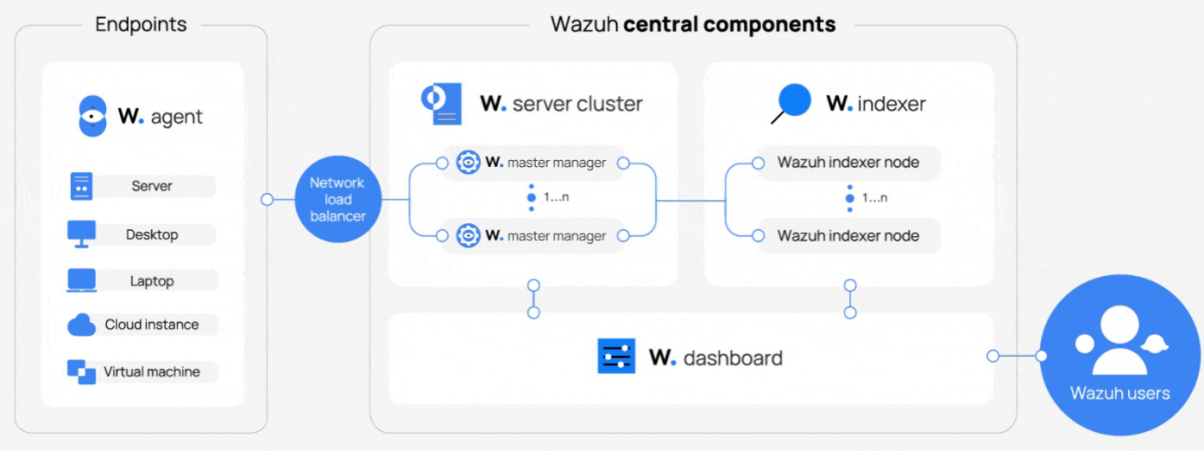


Figura 3: Exemplo de Componentes Funcionamento Wazuh

4.1.1 Wazuh Manager

O Wazuh Manager[16] é o componente central responsável por receber e analisar os eventos enviados pelos agentes, identificando ameaças e comportamentos suspeitos em tempo real. A informação recolhida é processada através de decoders e regras, permitindo a geração de alertas de segurança. Este componente permite ainda a gestão centralizada dos agentes, incluindo o seu registo, autenticação e configuração remota, garantindo comunicações seguras. Os dados processados são posteriormente encaminhados para o Wazuh Indexer, onde ficam armazenados para pesquisa e análise.

De forma resumida, o Wazuh Manager integra várias funções essenciais:

- **Receção de eventos:** recebe os dados enviados pelos agentes instalados nos sistemas monitorizados;
- **Decoders:** interpretam os logs recebidos e extraem campos relevantes, como endereço IP, utilizador, serviço ou tipo de evento;
- **Motor de regras:** compara os eventos com regras de deteção para identificar comportamentos suspeitos ou violações de políticas;
- **Gestão de agentes:** permite registar, autenticar, configurar e acompanhar o estado dos agentes ligados ao servidor;
- **Encaminhamento de alertas:** envia os alertas e eventos processados para o Wazuh Indexer, onde ficam disponíveis para pesquisa e análise.

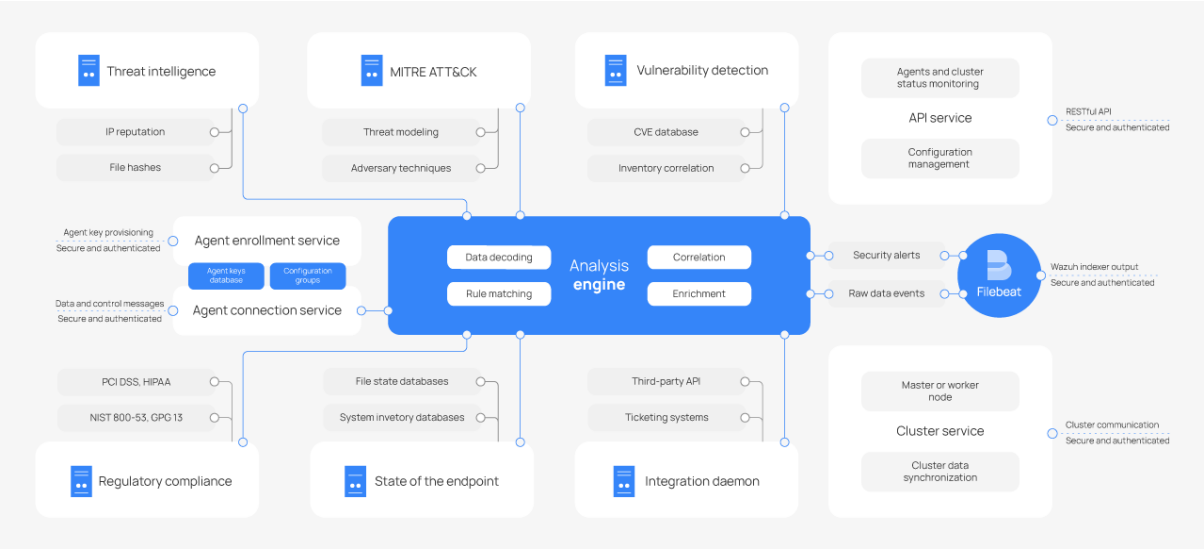


Figura 4: Wazuh Manager Architecture

4.1.2 Wazuh Indexer

O Wazuh Indexer[17] é o componente responsável por armazenar, indexar e organizar os dados gerados pelo Wazuh Server, permitindo a sua pesquisa e análise de forma eficiente. Os dados são armazenados como documentos JSON e organizados em índices.

Para garantir escalabilidade e alta disponibilidade, os índices são divididos em shards distribuídos por vários nós num cluster. Esta arquitetura permite distribuir a carga e assegurar redundância. O sistema opera quase em tempo real, tornando os dados pesquisáveis em cerca de um segundo após a indexação.

As suas principais componentes funcionais são:

- **Índices:** organizam os eventos e alertas em conjuntos de dados pesquisáveis;
- **Documentos JSON:** representam cada evento armazenado, facilitando a consulta estruturada da informação;
- **Shards:** dividem os índices em partes menores, permitindo distribuir os dados por vários nós;
- **Réplicas:** mantêm cópias dos dados para aumentar a disponibilidade e reduzir o risco de perda de informação;
- **Motor de pesquisa:** permite consultar rapidamente alertas, eventos, agentes e outros dados recolhidos pela plataforma.

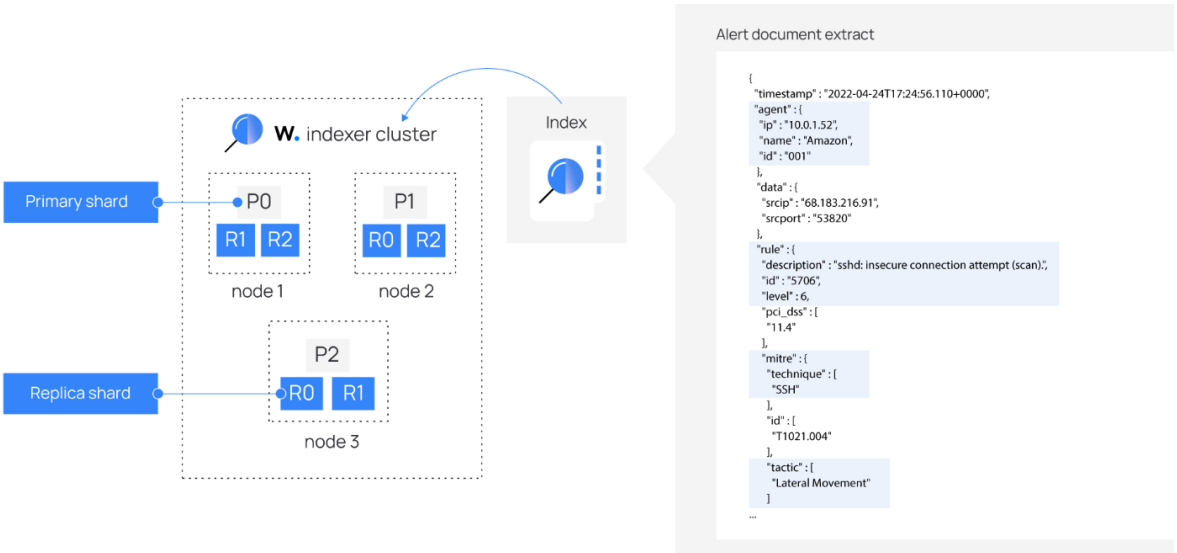


Figura 5: Arquitetura Indexer

4.1.3 Wazuh Dashboard

O Wazuh Dashboard[18] é a interface web gráfica que permite visualizar e analisar dados de segurança, como alertas, eventos, agentes e vulnerabilidades. Através desta interface, é possível monitorizar e gerir o estado da infraestrutura Wazuh de forma centralizada. O dashboard permite ainda configurar agentes, regras e outros componentes do sistema, bem como criar visualizações e relatórios personalizados. Integra-se com o Wazuh Server e o Wazuh Indexer, facilitando a investigação de eventos e a administração da plataforma.

As funcionalidades principais do Wazuh Dashboard incluem:

- **Visualização de alertas:** apresenta eventos de segurança de forma gráfica e organizada;
- **Consulta de agentes:** permite verificar o estado dos sistemas monitorizados e a sua ligação ao servidor;
- **Pesquisa de eventos:** possibilita filtrar e analisar dados armazenados no Wazuh Indexer;
- **Gestão da plataforma:** disponibiliza opções de administração, configuração e acompanhamento do ambiente Wazuh;
- **Relatórios e dashboards:** permite construir painéis e visualizações para apoiar a análise de segurança.

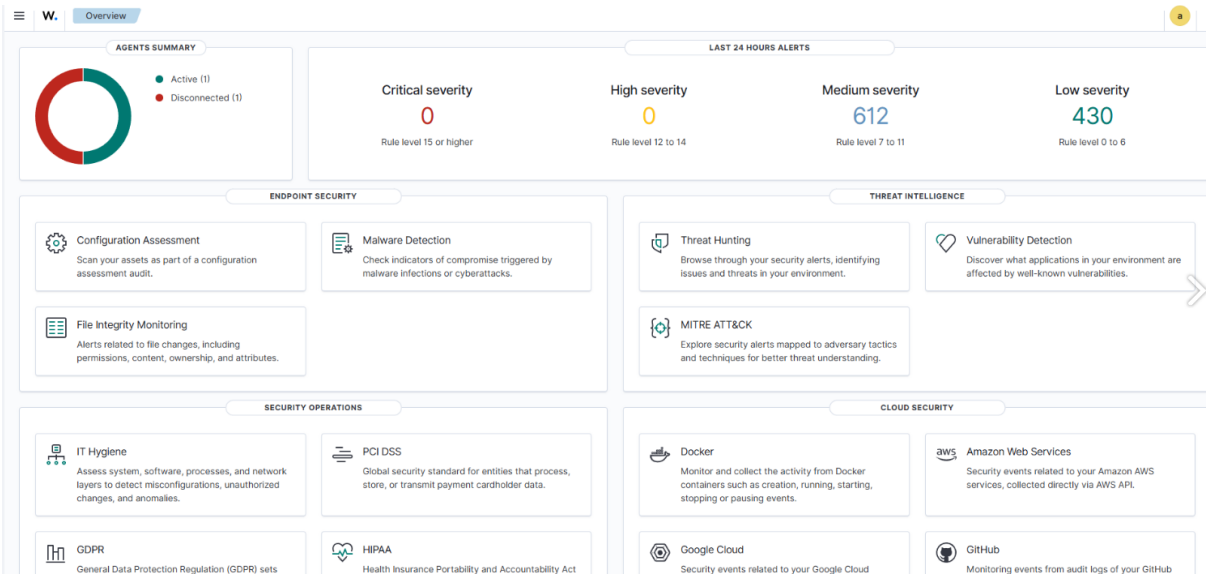


Figura 6: Wazuh Dashboard Inteface

4.1.4 Wazuh Agent

O Wazuh Agent[19] é instalado nos sistemas a monitorizar, sendo responsável pela recolha e monitorização de dados de segurança locais, como logs, eventos do sistema e alterações em ficheiros. Este componente segue uma arquitetura modular, permitindo executar diferentes tarefas de segurança, como monitorização de integridade, deteção de malware e análise de configurações. Os dados recolhidos são enviados de forma segura e autenticada para o Wazuh Server, onde são analisados. O agente pode ainda executar respostas automáticas a ameaças e ser gerido remotamente, adaptando-se a diferentes ambientes e necessidades de segurança (**Figura 7**)

O agente é composto por módulos responsáveis por diferentes tarefas locais:

- **Recolha de logs:** lê ficheiros de log e eventos do sistema operativo, enviando-os para o Wazuh Server;
- **Monitorização de integridade:** deteta alterações em ficheiros e diretórios importantes do sistema;
- **Inventário do sistema:** recolhe informação sobre software, hardware, processos e configurações locais;
- **Análise de vulnerabilidades e configuração:** ajuda a identificar software vulnerável e configurações inseguras;
- **Resposta ativa:** pode executar ações automáticas no host, como bloquear um endereço IP ou terminar uma ligação suspeita.

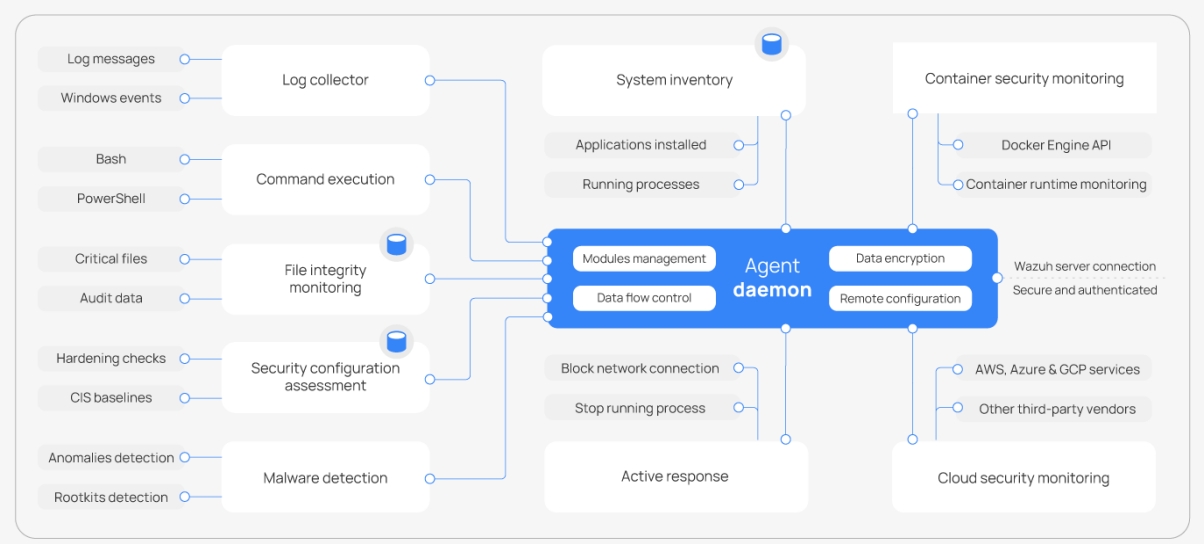


Figura 7: Wazuh Agent Architecture

4.2 Arquitetura

A arquitetura define a forma como os componentes do Wazuh são distribuídos. A escolha depende dos recursos disponíveis, da dimensão do ambiente e do nível de complexidade pretendido.

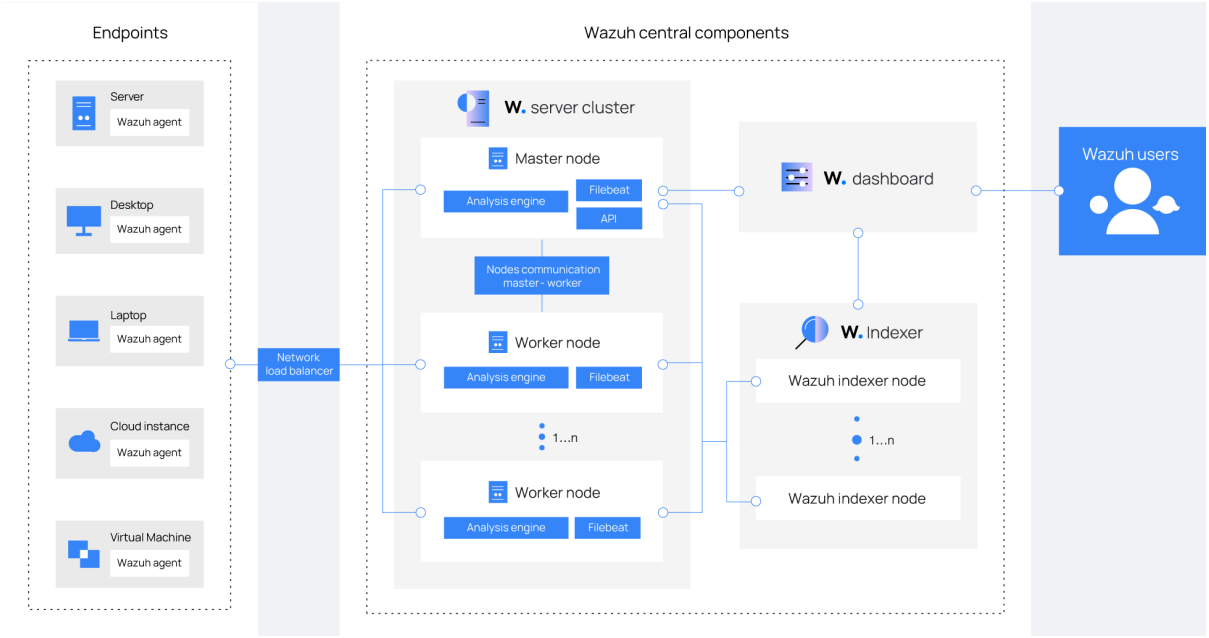


Figura 8: Arquitetura Wazuh

Os modelos de arquitetura existentes são o All-in-one, Single-Node e Multi-node. Todos possuem as mesmas componentes referidas no capítulo 4.1, variando a maneira de implementação dos mesmos tendo em conta o ambiente de produção pretentido.

4.2.1 All-in-one

Na arquitetura All-in-one(**Figura 9**), os principais componentes do Wazuh ficam instalados na mesma máquina, incluindo o Wazuh Server, o Wazuh Indexer e o Wazuh Dashboard. Neste modelo, todo o processo de recolha, análise, armazenamento e visualização dos eventos ocorre num único sistema.

Esta abordagem é indicada para ambientes laboratoriais, testes, demonstrações e projetos de menor dimensão, pois simplifica a instalação e a gestão da plataforma. Como todos os serviços se encontram concentrados no mesmo equipamento, a configuração inicial torna-se mais direta e exige menos recursos de infraestrutura.

No entanto, esta arquitetura apresenta limitações ao nível da escalabilidade e da disponibilidade. Caso a máquina onde o Wazuh está instalado fique indisponível, todos os serviços deixam de funcionar. Além disso, em ambientes com muitos agentes ou grande volume de eventos, o desempenho pode ser afetado, uma vez que todos os componentes competem pelos mesmos recursos de processamento, memória e armazenamento.

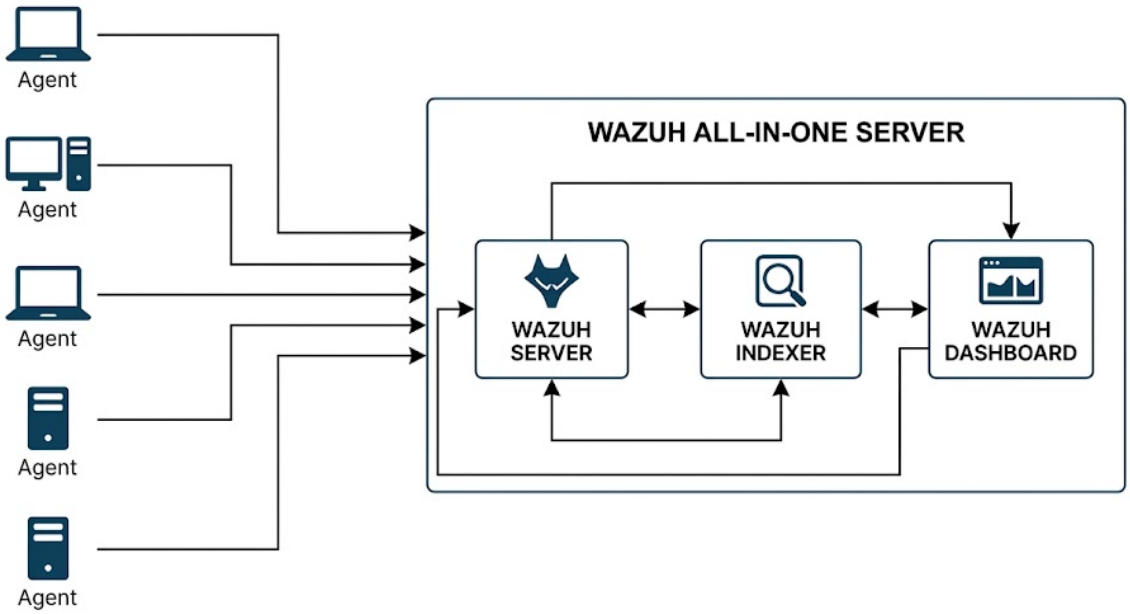


Figura 9: Arquitetura All-in-one

4.2.2 Single-node

Na arquitetura Single-node(**Figura 10**), os principais componentes do Wazuh são distribuídos por servidores separados. Assim, o Wazuh Server, o Wazuh Indexer e o Wazuh Dashboard são instalados individualmente em máquinas distintas, permitindo separar responsabilidades e reduzir a concorrência direta pelos mesmos recursos de processamento, memória e armazenamento [20].

Este modelo é recomendado para ambientes médios que necessitam de melhor desempenho do que uma instalação All-in-one. Ao separar os serviços por diferentes servidores, torna-se possível atribuir recursos específicos a cada componente, melhorando a capacidade de processamento, indexação e visualização dos eventos de segurança.

Apesar destas vantagens, a arquitetura Single-node não oferece a mesma escalabilidade e disponibilidade de uma arquitetura Multi-node. Cada componente continua a existir apenas numa instância, pelo que a falha de um dos servidores pode comprometer a função correspondente, como a recolha, a indexação ou o acesso ao dashboard. Ainda assim, representa uma solução intermédia entre a simplicidade do All-in-one e a robustez de uma implementação distribuída com vários nós.

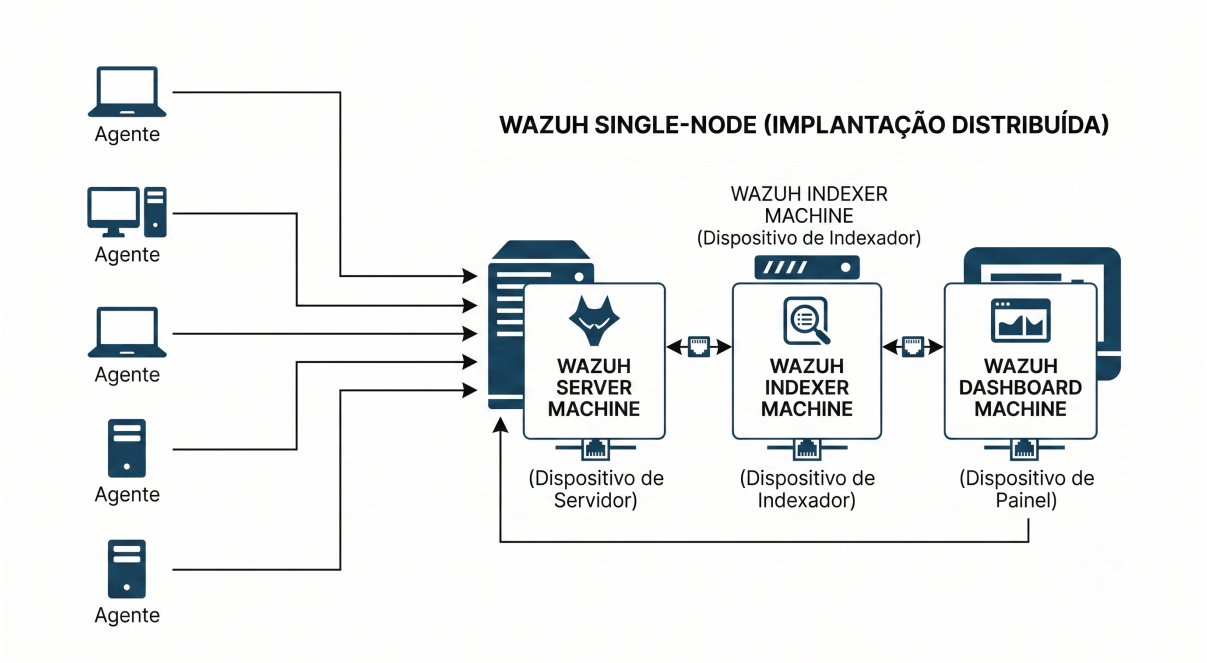


Figura 10: Arquitetura Single-Node

4.2.3 Multi-node

Na arquitetura Multi-node(**Figura 11**), os componentes do Wazuh são distribuídos por várias máquinas, permitindo separar responsabilidades e aumentar a capacidade da plataforma. Neste modelo, podem existir vários nós de Wazuh Server, vários nós de Wazuh Indexer e um ou mais pontos de acesso ao Wazuh Dashboard, dependendo da dimensão e dos requisitos da infraestrutura [20].

No caso do Wazuh Server, a arquitetura pode funcionar em modo cluster. Neste modo, os servidores Wazuh trocam informação entre si para manter consistência na gestão dos agentes, regras, configurações e estado operacional. O cluster é organizado com um nó principal, designado por *master node*, e um ou mais nós secundários, designados por *worker nodes*. O *master node* centraliza e sincroniza informação crítica do cluster, enquanto os *worker nodes* podem receber eventos de agentes e participar no processamento distribuído [21].

Esta separação permite distribuir a carga de trabalho. Em vez de todos os agentes comunicarem com um único servidor, estes podem ser distribuídos por diferentes nós Wazuh Server. Dessa forma, o processamento de eventos, a aplicação de regras e a gestão de ligações ficam repartidos, reduzindo a sobrecarga de um único sistema.

A arquitetura Multi-node é indicada para ambientes empresariais, infraestruturas críticas ou cenários com muitos agentes e grande volume de logs. As suas principais vantagens são a escalabilidade, a distribuição de carga e a melhoria da disponibilidade.

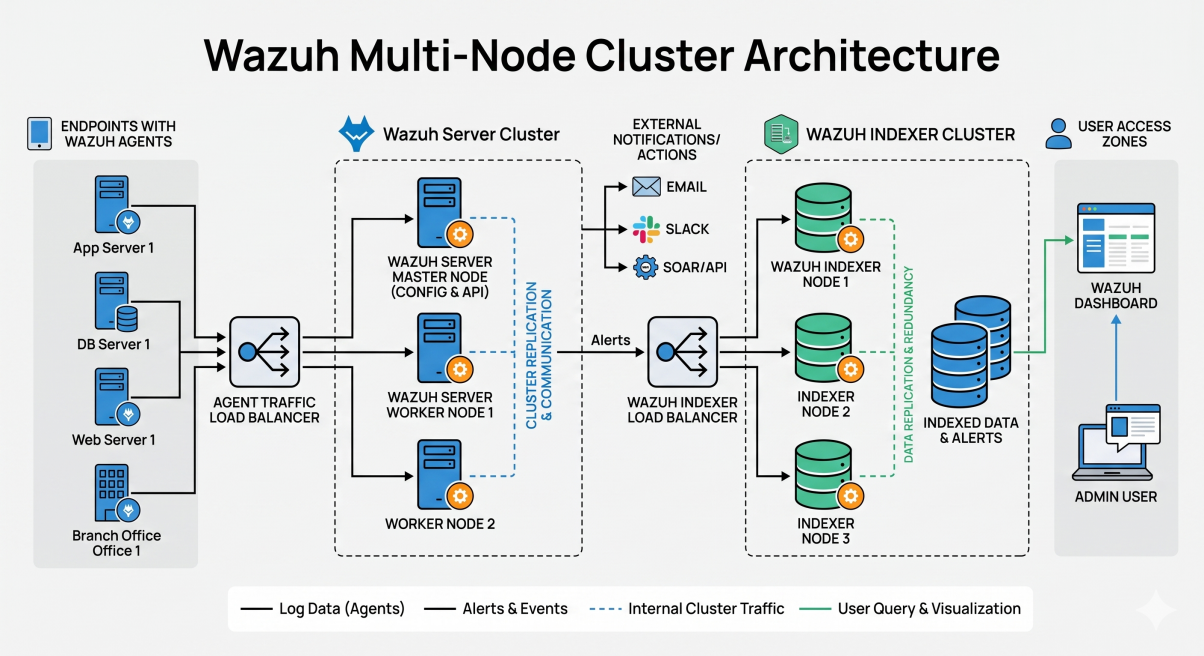


Figura 11: Arquitetura Multi-Node

5 Comunicação entre Componentes

O Wazuh utiliza diferentes canais de comunicação entre os seus componentes, dependendo da função desempenhada por cada serviço. De forma geral, a comunicação principal ocorre entre os agentes e o Wazuh Server, entre o Wazuh Server e o Wazuh Indexer, e entre a interface de visualização e os serviços que disponibilizam os dados para consulta. Cada uma destas ligações utiliza portos, mecanismos de autenticação e protocolos próprios, permitindo separar o envio de eventos, a gestão dos agentes, a indexação dos dados e a visualização da informação.

Para validar este modelo arquitetural descrito na documentação do Wazuh, a análise subsequente baseia-se também em observações do cenário configurado neste trabalho, ne inicialização da topologia montada e registo de um agente. A topologia é composta por um Wazuh Server (`192.168.1.1`) — configurado numa arquitetura *All-in-One* que aloja o Wazuh Manager, o Wazuh Indexer e o Wazuh Dashboard na mesma máquina, um Wazuh Agent (`192.168.1.2`) e uma *appliance* GNS3 Webterm (`192.168.1.3`) que atua como cliente de gestão através do seu navegador web, além de um router periférico (`192.168.1.254`) configurado para envio de telemetria Syslog.

5.1 Segurança na Comunicação

A comunicação deve garantir a confidencialidade, a integridade e a autenticação dos dados transmitidos. Estes princípios são importantes para evitar que os eventos sejam alterados, intercetados ou enviados por entidades não autorizadas. Por este motivo, é importante o estabelecimento de determinadas políticas que permitem a prevenção da realização de ataques maliciosos que podem pôr em causa a infraestrutura da rede.

5.1.1 Registo e Autenticação dos Agentes

Antes de um agente enviar eventos para o servidor, este deve ser registado no Wazuh Server. Este processo de *enrollment* utiliza, por omissão, o porto 1515/TCP [22] e permite ao servidor gerar uma chave única de sessão para o agente. Essa chave fica associada ao agente e passa a ser usada para autorizar a sua comunicação com o servidor [23], impedindo que sistemas não registados enviem eventos como se fossem agentes legítimos nos portos previstos para estes serviços [24].

O registo pode ser reforçado com opções adicionais de segurança. O Wazuh permite validar a identidade do Wazuh Manager através de certificados SSL/TLS assinados por uma autoridade certificadora [25], fazendo com que o agente confirme se está a registar-se no servidor correto. Também é possível validar a identidade do próprio agente através de certificados e chaves próprias [26]. Nestes cenários são usados certificados, chaves privadas e uma CA (Certification Authority), ou seja, mecanismos de criptografia assimétrica para autenticação das entidades envolvidas no processo de registo.

5.1.2 Comunicação Agente–Servidor

Após o registo, o Wazuh Agent envia os eventos recolhidos no host para o Wazuh Server através do serviço de ligação de agentes, normalmente no porto 1514/TCP [22]. A utilização de UDP também é possível, mas encontra-se desativada por omissão. Esta ligação transporta os eventos que serão posteriormente descodificados e analisados pelo motor de regras do Wazuh Manager [20], usando os portos previstos pela documentação oficial [24].

Nesta comunicação, o Wazuh utiliza cifra simétrica [20]. Por omissão, o protocolo de mensagens entre agente e servidor usa AES com blocos de 128 bits e chaves de 256 bits, o Blowfish existe apenas como opção alternativa [27]. O AES funciona em modo CBC, recorrendo a um vetor de inicialização para evitar que mensagens iguais produzam blocos cifrados repetidos.

No tráfego de rede analisado este comportamento é visível quando o Wazuh Agent (`192.168.1.2`) inicia o *three-way handshake* TCP a partir do seu porto dinâmico `51189` em direção ao porto `1514` do Wazuh Server (`192.168.1.1`). Após a finalização do aperto de mão, os pacotes subsequentes demonstram a transferência ativa de dados cifrados (*flags* `[PSH, ACK]`), confirmando a transmissão contínua de alertas e mensagens de *keep-alive*.

5.1.3 Comunicação com Indexer, Dashboard e API

Depois da análise no Wazuh Manager, os alertas e eventos são encaminhados para o Wazuh Indexer [17]. Esta comunicação é realizada pelo Filebeat e utiliza TLS para proteger o envio dos dados até ao serviço do Indexer, normalmente disponível no porto 9200/TCP [24]. O Wazuh Dashboard [18] consulta os dados indexados e comunica também com o Wazuh API, normalmente no porto 55000/TCP [24], para obter informação de gestão, como o estado dos agentes e a configuração do servidor.

Esta interação segura pode ser observada quando o cliente Webterm (`192.168.1.3`) estabelece uma ligação direta com o serviço web do Wazuh Server (`192.168.1.1`) no porto `443`. Numa instalação *All-in-One*, este porto centraliza o acesso seguro à interface gráfica do Dashboard e às respetivas APIs da infraestrutura.

Para garantir a confidencialidade desta comunicação cliente-servidor, a ligação utiliza o protocolo **TLSv1.3**. Quando o cliente HTTPS (`192.168.1.3`) contacta o servidor (`192.168.1.1`), ambas as partes negoceiam os parâmetros de segurança e estabelecem uma chave de sessão simétrica (este processo é visível na rede através das mensagens de *Server Hello* e *Change Cipher Spec*). A partir do momento em que esta fase de autenticação e troca de chaves é concluída, todos os dados transferidos passam a ser encapsulados, surgindo na captura de rede simplesmente como *Application Data*. O TLSv1.3 assegura, assim, que informações sensíveis como credenciais de acesso, consultas efetuadas pelo utilizador ou o conteúdo dos alertas, fiquem totalmente cifradas e ilegíveis para qualquer entidade que intercete os PDUs.

5.1.4 Receção de Logs Externos (Agentless e Syslog)

Para além da monitorização nativa baseada em agentes, o Wazuh Server pode atuar como um servidor de *syslog* centralizado, permitindo a monitorização *agentless* de ativos da rede que suportem o protocolo Syslog.

A análise do tráfego confirma esta capacidade de receção de logs, no qual o router envia mensagens de diagnóstico em direção ao Wazuh Server.

O servidor integra os registos do Syslog, onde as notificações geradas pelo router descrevem os seus eventos e ações. Estes pacotes Syslog são enviados para o porto UDP padrão (`514`) no Wazuh Server, o que permite processar e normalizar telemetria externa em tempo real sem depender de agentes dedicados no host.

5.1.5 Comunicação em Cluster e Portos Relevantes

Em arquiteturas Multi-node, os nós do Wazuh Server podem comunicar entre si através do serviço de cluster, normalmente no porto 1516/TCP [24]. Esta comunicação permite sincronizar informação interna entre nós, mantendo a consistência na gestão dos agentes e na operação distribuída [20]. O Wazuh também define outros portos relevantes, como o 9300–9400/TCP para comunicação interna entre nós do cluster do Wazuh Indexer [24].

De forma resumida, considerando o mapeamento teórico e as evidências práticas recolhidas na captura de rede, os portos mais importantes na rede implementada são:

Porto	**Protocolo**	**Finalidade**
514	UDP/TCP	Receção de mensagens de auditoria via Syslog (*Agentless*).
1514	TCP	Comunicação ativa e envio de eventos do Wazuh Agent para o Wazuh Server.
1515	TCP	Registo e autenticação inicial dos agentes (*Enrollment*).
1516	TCP	Comunicação entre nós do cluster do Wazuh Server.
9200	TCP	Acesso à API do Wazuh Indexer (comunicação restrita/autenticada).
9300–9400	TCP	Comunicação interna entre nós do cluster do Indexer.
55000	TCP	Acesso à API de gestão interna do Wazuh Server.
443	TCP	Acesso seguro exposto pelo servidor para a interface web do Dashboard.

Tabela 2: Portos principais utilizados na comunicação da infraestrutura Wazuh analisada

Estes portos devem ser permitidos apenas entre os componentes que realmente necessitam de comunicar, reduzindo significativamente a superfície de ataque da infraestrutura de segurança.

6 Setup e Configuração da Topologia

Este capítulo apresenta a organização geral do setup utilizado para a realização das experiências propostas. A descrição detalhada dos procedimentos de instalação, preparação e configuração foi transferida para os anexos, de forma a manter o corpo principal do relatório mais direto e a concentrar os comandos técnicos em secções próprias de consulta.

Para executar a topologia, é necessário ter previamente instalados os recursos indicados na **secção 2.2** e compreender a arquitetura descrita na **secção 4.2**. A preparação do ambiente segue uma sequência lógica: conversão das imagens, associação das máquinas virtuais ao GNS3, configuração do router, configuração dos serviços de rede e, por fim, associação do agente ao servidor Wazuh.

Os procedimentos completos encontram-se organizados nos seguintes anexos:

Anexo A Conversão das imagens das máquinas virtuais para o formato QEMU/*qcow2*.

Anexo B Criação e associação dos templates das máquinas virtuais no GNS3.

Anexo C Configuração do router, incluindo interfaces, NAT [28], ACLs e DHCP [29].

Anexo D Preparação da rede, configuração de IP estático e atualização do servidor Wazuh [30].

Anexo E Instalação e associação do agente Wazuh ao servidor.

Anexo F Configuração dos serviços SSH e Apache [31].

Anexo G Configuração do Telnet no servidor e no agente Wazuh.

Anexo H Consulta de comandos e definições.

Anexo I Script Python utilizado no ambiente laboratorial.

Anexo J Ferramentas utilizadas no Kali Linux.

Desta forma, o capítulo mantém o enquadramento do setup da topologia, enquanto os anexos funcionam como guias técnicos independentes e ordenados para replicação do ambiente laboratorial.

7 Sistemas IDS e IPS

Este capítulo apresenta os conceitos de IDS e IPS e explica a forma como estes se relacionam com as políticas de segurança utilizadas no laboratório. O objetivo é separar a componente teórica da componente experimental, permitindo que os testes realizados posteriormente tenham uma base mais clara.

7.1 Conceitos de IDS e IPS

Um sistema IDS (*Intrusion Detection System*) tem como principal função detetar atividades suspeitas e gerar alertas. Este tipo de sistema não bloqueia diretamente a ameaça, mas permite identificar comportamentos anómalos e apoiar a análise de segurança.

Um sistema IPS (*Intrusion Prevention System*) tem uma função mais ativa, pois, além de detetar comportamentos suspeitos, pode aplicar medidas de prevenção ou bloqueio. Desta forma, o IPS atua como uma camada adicional de proteção, impedindo ou limitando determinadas ações maliciosas.

7.2 Diferenças entre IDS e IPS

A principal diferença entre um sistema IDS e um sistema IPS está no tipo de resposta. Enquanto o IDS está focado na deteção e geração de alertas, o IPS está associado à prevenção e à reação automática perante determinadas ameaças ou alertas.

No contexto deste projeto, o Wazuh será utilizado para observar eventos, gerar alertas e apoiar a análise de comportamentos suspeitos. A componente IPS será abordada através da criação de políticas e da avaliação de respostas automáticas perante os eventos detetados.

8 Sistemas XDR e HIDS

8.1 XDR

O termo XDR (*Extended Detection and Response*), refere-se a uma abordagem de segurança que procura integrar dados provenientes de várias fontes da infraestrutura, como endpoints, servidores, redes, aplicações e serviços cloud. Em vez de analisar cada componente de forma isolada, uma solução XDR centraliza os eventos, correlaciona indicadores de ataque e apresenta uma visão mais abrangente do incidente. Esta abordagem facilita a deteção de ameaças complexas, pois permite relacionar atividades suspeitas que, individualmente, poderiam parecer pouco relevantes.

8.2 HIDS

O HIDS (Host-based Intrusion Detection System), é um sistema de deteção de intrusões baseado no próprio equipamento monitorizado. Ao contrário de mecanismos focados apenas no tráfego de rede, um HIDS observa o comportamento local do sistema, analisando logs, ficheiros, processos, autenticações, alterações de configuração e outros eventos gerados no host. Esta abordagem é particularmente útil para identificar alterações suspeitas em ficheiros, tentativas de autenticação indevidas, execução de comandos anómalos ou modificações não autorizadas no sistema.

8.3 Diferenças entre XDR e HIDS

A principal diferença entre XDR e HIDS está no âmbito de atuação. O HIDS concentra-se na monitorização de um host específico, analisando eventos locais desse sistema. O XDR tem uma visão mais ampla, agregando e correlacionando eventos de vários hosts e componentes da infraestrutura. Assim, o HIDS pode ser visto como uma das fontes de informação que alimenta uma arquitetura XDR. No caso do Wazuh, os agentes fornecem a visibilidade local típica de um HIDS, enquanto a plataforma central permite correlacionar esses dados e aplicar mecanismos de deteção e resposta mais abrangentes.

9 Aplicação de Políticas de Segurança

Este capítulo apresenta a configuração das políticas de segurança aplicadas aos sistemas do laboratório, recorrendo às capacidades IDS/IPS do Wazuh. O objetivo é definir estratégias de defesa adequadas à superfície de ataque explorada e validar a eficácia das respostas configuradas.

9.1 Bloqueio de um IP malicioso (Serviços Web)

Nesta secção, o agente Xubuntu será protegido pelo servidor Wazuh contra tentativas de interação Web provenientes de um IP malicioso identificado. O objetivo desta política é garantir uma resposta de bloqueio total ao nível da rede, impedindo qualquer tipo de comunicações entre o atacante e o agente depois da primeira interação HTTP [32].

O primeiro passo consiste em utilizar uma lista de reputação (CDB List) para indicar ao Wazuh que o IP do Kali Linux é malicioso e deve ser monitorizado. Esta configuração é realizada no Wazuh Server (192.168.1.1).

O Wazuh já inclui uma lista de reputação para IPs maliciosos no seu conjunto de regras padrão, conforme apresentado na **Figura 12**:

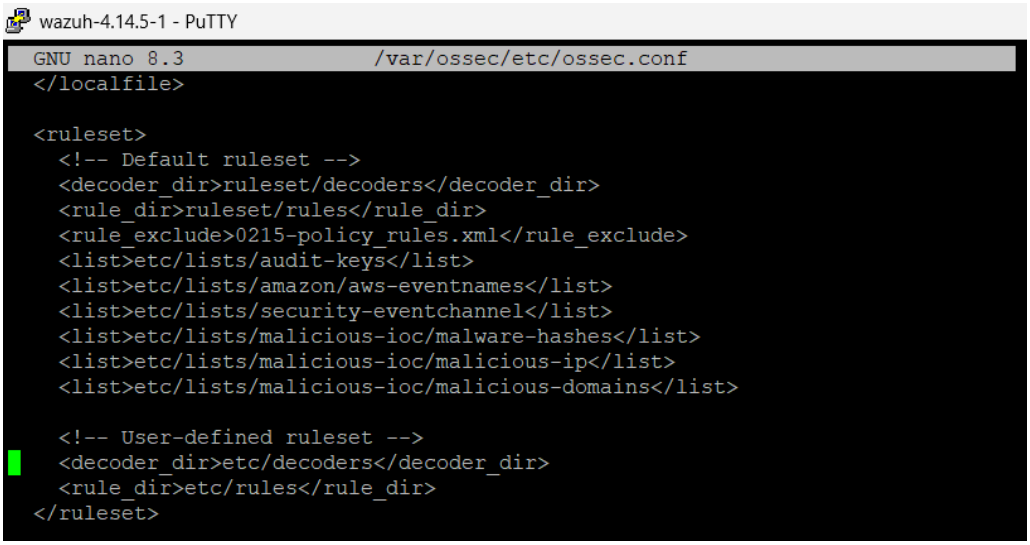


Figura 12: Lista de reputação presente no conjunto de regras do Wazuh

9.1.1 Adição de IP do Kali à lista de ips maliciosos

```
sudo -i
sudo nano /var/ossec/etc/lists/malicious-ioc/malicious-ip
```

Listing 9.1: Edição da lista de IPs maliciosos

É obrigatório utilizar a sintaxe "**EndereçoIP:**"", como presente na **Figura 13**

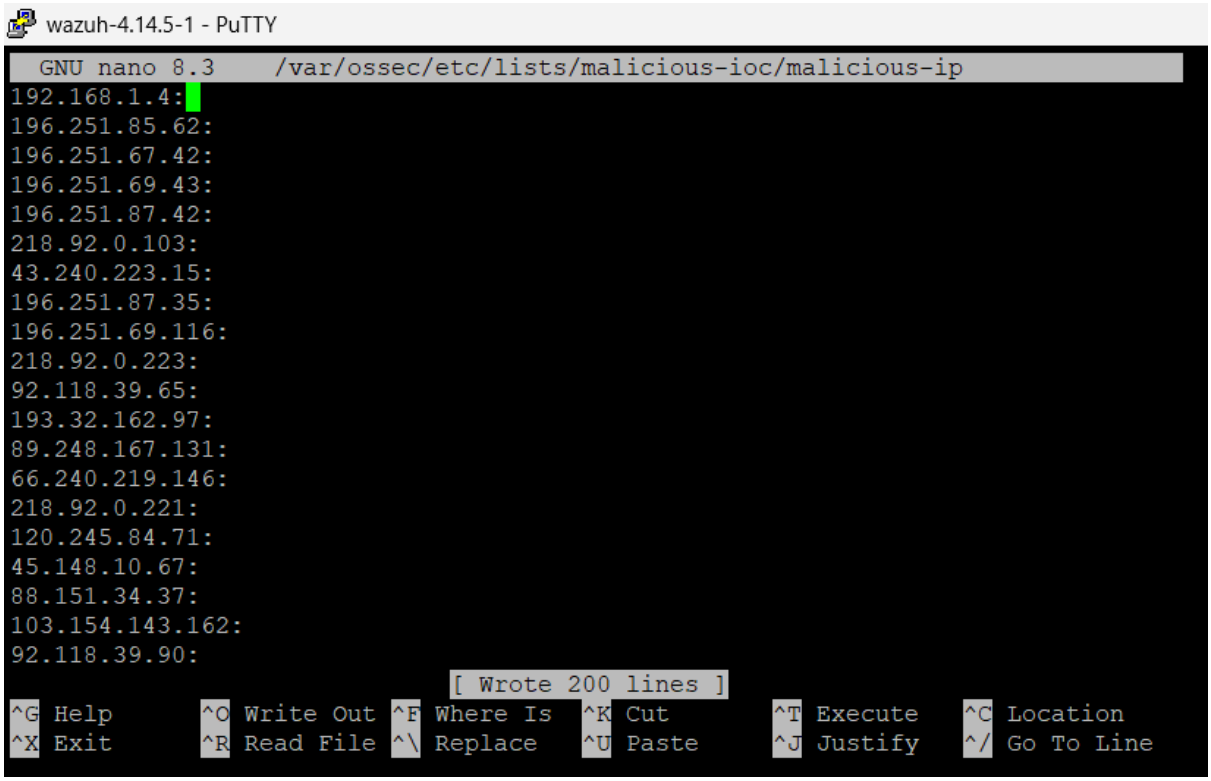


Figura 13: Lista de IPs maliciosos configurada no Wazuh

Esta lista já se encontra previamente declarada no ficheiro de configuração central do Wazuh para ser verificada (*ossec.conf*).

9.1.2 Criação da Regra de Deteção

As categorias ou grupos são definidos pelas regras padrão do Wazuh. Estas regras estão definidas em vários ficheiros no formato XML habitual. Estes ficheiros existem por omissão na configuração do Wazuh e podem ser consultados usando o comando *ls -la /var/ossec/ruleset/rules/*.

Para adicionar a regra, edita-se o ficheiro de regras locais do Wazuh Server (**9.2**):

```
sudo nano /var/ossec/etc/rules/local_rules.xml
```

Listing 9.2: Edição das regras locais do Wazuh

Adiciona-se o seguinte bloco de código XML no final do ficheiro (**9.3**):

```
<group name="local, ip_reputation,">
  <rule id="100100" level="10">
    <if_group>web|attack|attacks|accesslog</if_group>
    <list field="srcip" lookup="address_match_key">etc/lists/malicious-ioc/
   malicious-ip</list>
    <description>Acesso Web bloqueado de um IP malicioso conhecido: $(srcip)
   .</description>
  </rule>
</group>
```

Listing 9.3: Regra local para bloqueio de IP malicioso

A tag <*if_group*> serve para adicionar os grupos que contêm as regras já definidas pelo Wazuh á regra local configurada e atua como um filtro de logs recebidos. O Wazuh possui milhares de regras embutidas, e processar todos os logs contra listas personalizadas consumiria recursos excessivos. Este filtro instrui o servidor a verificar a lista de IPs maliciosos apenas se o log pertencer a um evento relevante da rede ou sistema configurado nos grupos.

9.1.3 Configurar o Sistema de Prevenção (Active Response):

Para transformar a deteção (IDS) numa ação preventiva (IPS), é necessário que o sistema reaja à regra *100100* cortando a comunicação. Edita-se o ficheiro *ossec.conf* do Wazuh Server (**9.4**):

```
sudo nano /var/ossec/etc/ossec.conf
```

Listing 9.4: Edição do ficheiro ossec.conf

Procura-se a secção dedicada a <*active-response*> (ou adiciona-se antes do fecho da tag <*ossec_config*>) e insere-se a seguinte configuração (**9.5**):

```
<active-response>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>100100</rules_id>
  <timeout>180</timeout>
</active-response>
```

Listing 9.5: Configuração da resposta ativa do Wazuh

- **active-response:** Define o bloco de configuração responsável por associar um alerta do Wazuh a uma ação automática de resposta. O Wazuh deixa de atuar apenas como IDS, que deteta e alerta, e passa também a executar uma ação preventiva (IPS).
- **command:** Invoca o script nativo do Wazuh que interage com o *iptables* do Linux. É importante notar que este comando bloqueia o tráfego a nível de rede (Camada 3 do Modelo OSI). Ao ser ativado, a firewall descarta silenciosamente qualquer pacote IP proveniente do IP atacante, independentemente de protocolos de níveis superiores [30].
- **location:** Define que o script será executado no modo local (no agente onde a ameaça foi detetada, o Xubuntu).
- **timeout:** Mantém a regra de bloqueio da firewall ativa durante 180 segundos. A primeira tentativa de ligação pelo atacante irá despoletar a resposta ativa. Tentativas subsequentes durante este período resultarão num esgotamento do tempo de resposta (timeout).
- **rules_id:** Indica o identificador da regra que deve acionar a resposta ativa. Para esta política foi usado o ID *100100*, correspondente à regra local criada para detetar acessos Web provenientes de IPs presentes na lista de reputação maliciosa. Assim, a ação *firewall-drop* só é executada quando o alerta gerado corresponde a esta regra específica, evitando bloqueios indevidos provocados por outros eventos do sistema.

Sendo o Wazuh um HIDS/HIPS reativo orientado a logs, o estabelecimento inicial da comunicação (*TCP handshake*: SYN -> SYN-ACK -> ACK) [22] chegará a ocorrer e normalmente o primeiro pedido HTTP será respondido [32]. A tag <*list*> verifica a variável *srcip* (IP de origem) face à lista de IPs indicada na regra. Ao detetar uma correspondência, o alerta de nível 10 é gerado e o agente é indicado a executar o script nativo *firewall-drop*.

Por omissão, o servidor Apache instalado no agente regista os seus logs no ficheiro local */var/log/apache2/access.log*. Para que o Wazuh server consiga processar estes eventos e gerar os alertas associados aos grupos indicados é fundamental garantir que o agente lê e envia os conteúdos deste ficheiro para o servidor. (**9.6**).

```
sudo nano /var/ossec/etc/ossec.conf
```

Listing 9.6: Ficheiro de configuração do agente

Foi adicionado o seguinte bloco na secção <*ossec_config*> para definir o formato e localização de logs locais (**9.7**):

```
<localfile>
  <log_format>apache</log_format>
  <location>/var/log/apache2/access.log</location>
</localfile>
```

Listing 9.7: Configuração da recolha de logs do Apache

Por fim, reinicia-se o agente para aplicar as alterações (**9.8**):

```
sudo systemctl restart wazuh-agent
```

Listing 9.8: Reinício do agente Wazuh

9.2 Bloqueio dinâmico (SSH e Telnet)

Esta secção é focada na proteção de serviços de administração remota (SSH [31] e Telnet [33]) contra ataques de força bruta (*brute force*), aplicável a qualquer endereço IP.

Neste caso adotou-se uma política de segurança comportamental ou condicional. Em vez de bloquear o acesso de forma imediata na tentativa de uma ligação de um IP específico, o servidor permite algumas falhas nas credenciais de acesso (assumindo erros do utilizador), mas bloqueia imediatamente o IP de origem depois de sinais de um ataque de força bruta.

O Wazuh possui regras nativas de correlação temporal para identificar estes ataques. Analisando o *ruleset* padrão, identificaram-se as seguintes regras chave de força bruta:

- **Regra 5712:** Ataque de força bruta SSH recorrendo a utilizadores inexistentes [31]. Esta regra está definida no ficheiro */var/ossec/ruleset/rules/0095-sshd_rules.xml*.
- **Regra 5763:** Ataque de força bruta SSH recorrendo a utilizadores válidos [31] (falhas repetidas de *password*). Esta regra está definida no ficheiro */var/ossec/ruleset/rules/0095-sshd_rules.xml*.
- **Regra 5551:** Ataque de força bruta a módulos PAM (aplicável ao serviço Telnet [33]). Esta regra está definida no ficheiro */var/ossec/ruleset/rules/0085-pam_rules.xml*.

Estas regras foram adicionadas á resposta ativa do Wazuh criada anteriormente, mapeando o script *firewall-drop* e às regras de *brute force* (**9.9**).

```
sudo nano /var/ossec/etc/ossec.conf
```

Listing 9.9: Edição do ficheiro ossec.conf

```
<active-response>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>100100,5712,5763,5551</rules_id>
  <timeout>180</timeout>
</active-response>
```

Listing 9.10: Configuração da resposta ativa do Wazuh

Por fim, reinicia-se o serviço do gestor Wazuh para aplicar as novas políticas (**9.11**):

```
sudo systemctl restart wazuh-manager
```

Listing 9.11: Reinício do Wazuh Manager

9.3 Proteção contra *SYN floods* & *Port Scans*

Nesta secção, as interfaces do router ligadas ás duas redes privadas serão protegidas contra ataques de varrimento de portos (*Port Scans*) e *SYN Floods*. A rede 192.168.1.0/24 e o agente também serão protegidos contra estes ataques vindos da rede 192.168.2.0/24. Serão estabelecidas políticas de segurança IPS dinâmicas, de forma a prevenir que o ataque afete tanto os equipamentos de rede como os sistemas terminais.

9.3.1 Proteção e configuração do Router

A primeira linha de defesa assenta na configuração de uma lista de controlo de acesso (ACL) chamada *MASTER_POLICY* no router Cisco. O objetivo desta ACL é atuar como um sensor da rede e fazer a distinção entre os dois tipos de ataque.

A ACL foi estruturada com a seguinte lógica:

- **Ligação de gestão:** Permite a comunicação via Telnet exclusivamente entre o servidor Wazuh e o router [33], essencial para a automação da mitigação dos ataques através da troca de logs (sentido router → Wazuh) e comandos IOS (sentido Wazuh → router).
- **Monitorização *SYN Flood*:** Permite o tráfego HTTP legítimo direcionado ao router e às redes [32], mas obriga à geração de um registo (log) por cada pacote que passe no router. Deste modo o ataque pode ser detetado com destino no router ou na rede protegida.
- **Monitorização *Port Scan*:** Bloqueia proativamente e gera um log para qualquer tentativa de acesso a portos não autorizados no router ou no agente protegido (Xubuntu), atuando como um sensor para detetar *scans* com destino no router ou no agente.
- **Tráfego Legítimo:** Permite a livre passagem do restante tráfego sem gerar logs que não coincida com as regras anteriores.

As regras são então aplicadas às interfaces do router que ligam entre a rede protegida e a rede desprotegida.

```
ip tcp intercept mode intercept
ip access-list extended MASTER_POLICY
 permit tcp host 192.168.1.1 host 192.168.1.254 eq 23
 permit tcp any host 192.168.1.254 eq 80 log
 permit tcp any host 192.168.2.254 eq 80 log
 permit tcp any any eq 80 log
 deny tcp 192.168.2.0 0.0.0.255 host 192.168.1.2 log
 deny tcp any host 192.168.1.254 log
 deny udp any host 192.168.1.254 log
 deny tcp any host 192.168.2.254 log
 deny udp any host 192.168.2.254 log
 permit ip any any
exit
interface Ethernet0/0
 ip access-group MASTER_POLICY in
 exit
interface Ethernet0/2
 ip access-group MASTER_POLICY in
 exit
```

Listing 9.12: Configuração do router e ACL como sensor de rede

9.3.2 Descodificador personalizado para o Syslog

Quando o router gera os logs das ACLs, estes são colocados num datagrama UDP e enviados via Syslog [34] para o servidor Wazuh. É imperativo que o Wazuh consiga interpretar estes textos brutos antes de os poder avaliar.

O fluxo de análise do Wazuh dita que os logs passam obrigatoriamente primeiro pelos descodificadores. O descodificador *cisco-ips-custom* deteta a palavra-chave "SEC-6-IPACCESSLOG"na mensagem bruta e assume a responsabilidade do seu processamento.

Através de expressões regulares (Regex), o descodificador extrai os endereços IP criando variáveis dinâmicas em memória [30] (*srcip* e *dstip*). Embora o script de resposta automática dependa apenas do IP de origem para isolar o atacante, a extração do IP de destino (*dstip*) é fundamental para enriquecer a telemetria do sistema, permitindo aos administradores identificar visualmente no painel (Dashboard) qual a máquina específica que esteve sob ataque.

```
sudo nano /var/ossec/etc/decoders/local_decoder.xml

<decoder name="cisco-ips-custom">
  <prematch>%SEC-6-IPACCESSLOG</prematch>
</decoder>

<decoder name="cisco-ips-custom-fields">
  <parent>cisco-ips-custom</parent>
  <regex>tcp (\d+.\d+.\d+.\d+)\(\d+\) -> (\d+.\d+.\d+.\d+)\(\d+\)</regex>
  <order>srcip, dstip</order>
</decoder>
```

Listing 9.13: Criação do descodificador personalizado para o Cisco IOS

9.3.3 Criação de Regras de Correlação Temporal no Wazuh

Uma vez descodificado e estruturado, o log avança para o motor de regras do Wazuh. A arquitetura foi dividida numa lógica de hierarquia (pai-filho):

A **Regra PAI 100300** valida o evento exclusivamente se este tiver sido processado pelo descodificador correto e contiver o termo secundário da ACL. Com a regra pai ativa, o motor avalia as regras filho baseando-se na ação registada pelo router:

- Se a mensagem contiver a palavra **"denied"**, o fluxo aciona a regra **100301** (**scan de Portos**).
- Se a mensagem contiver a palavra **"permitted"**, o fluxo aciona a regra **100302** (**SYN Flood**).

Ambas as regras filho possuem correlação temporal. O Wazuh armazena o evento em memória e só desencadeia o alarme (Nível 10 ou 12) se detetar tráfego repetido dentro de uma janela de tempo definida de forma a evitar falsos positivos.

```
sudo nano /var/ossec/etc/rules/local_rules.xml

<group name="cisco,ios,ips,">
  <rule id="100300" level="3">
    <decoded_as>cisco-ips-custom</decoded_as>
    <match>IPACCESSLOG</match>
    <description>Cisco syslog: trafego apanhado na ACL.</description>
  </rule>

  <rule id="100301" level="12" frequency="2" timeframe="60">
    <if_matched_sid>100300</if_matched_sid>
    <match>denied</match>
    <description>IPS: Possivel scan de portos detetado. A ativar Null Route!</
    description>
    <mitre><id>T1046</id></mitre>
  </rule>

  <rule id="100302" level="10" frequency="10" timeframe="10">
    <if_matched_sid>100300</if_matched_sid>
    <match>permitted</match>
    <description>IPS: Possivel SYN Flood detetado. A ativar TCP Intercept!</
    description>
    <mitre><id>T1498</id></mitre>
  </rule>
</group>
```

Listing 9.14: Criação de regras para deteção de Scans e Floods

9.3.4 Automação de resposta IPS (Script Python *cisco_ips.py*)

Quando o alarme de Nível 12 (Scan) ou Nível 10 (Flood) é disparado, o módulo do *Active Response* executa o script *cisco_ips.py*. O script completo, cujo código fonte se encontra nos anexos deste relatório, encarrega-se de executar o ciclo de defesa através dos seguintes passos:

1. **Handshake Stateful active response (Wazuh → Python):** O Wazuh injeta os dados do ataque (em JSON) na entrada padrão (STDIN) do script. O script extrai o IP malicioso e pede autorização ao Wazuh (*check_keys*) para intervir. Se o IP ainda não estiver bloqueado, o Wazuh responde com "continue"para impedir repetições de ações desnecessárias.
2. **Ligação Netmiko:** O script utiliza a biblioteca Python *Netmiko* para estabelecer uma ligação Telnet automática com o router Cisco [33], introduzindo as credenciais de gestão.
3. **Lógica de Mitigação:** O script avalia a variável *rule_id* para injetar a defesa correta no router:
   - **Regra 100301 (Port Scan):** Injeta uma rota nula (*ip route <IP> 255.255.255.255 Null0*). Esta abordagem bloqueia o IP atacante de forma cirúrgica, uma vez que a origem do ataque está confirmada.
   - **Regra 100302 (SYN Flood):** Em ataques volumétricos, os IPs de origem costumam ser forjados (spoofing), tornando o bloqueio por IP ineficaz. Neste caso, o script invoca a defesa nativa do router *ip tcp intercept list SYN_PROTECT*, introduzindo limiares agressivos (*max-incomplete low 50 high 100*) para limpar rapidamente as conexões falsas da memória.
4. **Gestão de Timeout:** Concluído o tempo de bloqueio, o Wazuh reexecuta o script enviando a indicação *command: "delete"*. O Python volta a ligar-se ao router por Telnet [33] para retirar as rotas estáticas ou as regras de interceção TCP, devolvendo o router ao seu estado normal (*statefull active response*).

Para garantir a execução correta do programa em Python pelo sistema operativo, devem ser aplicadas permissões restritas ao script de resposta:

```
sudo chmod 750 /var/ossec/active-response/bin/cisco_ips.py
sudo chown root:wazuh /var/ossec/active-response/bin/cisco_ips.py
```

Listing 9.15: Permissões de segurança do script Python

9.3.5 Registo e ativação do comando de resposta ativa

O último passo consiste em registar e associar o script aos IDs das regras, determinando os respetivos intervalos de tempo deste que o script é ativado até ser reativado para reverter os comandos no router. Configurou-se um bloqueio de 10 minutos (600 segundos) para os *port scans* e para a interceção de pacotes TCP.

```
sudo nano /var/ossec/etc/ossec.conf

<command>
    <name>cisco-ips</name>
    <executable>cisco_ips.py</executable>
    <timeout_allowed>yes</timeout_allowed>
  </command>

  <active-response>
    <command>cisco-ips</command>
    <location>server</location>
    <rules_id>100301</rules_id>
    <timeout>600</timeout>
  </active-response>

  <active-response>
    <command>cisco-ips</command>
    <location>server</location>
    <rules_id>100302</rules_id>
    <timeout>600</timeout>
  </active-response>
```

Listing 9.16: Associação das regras à Active Response

Finalmente, o sistema é compilado reiniciando o serviço do gestor Wazuh que trata de todos os mecanismos e ficheiros configurados anteriormente.

```
sudo grep -i -E "error|xml" /var/ossec/logs/ossec.log | tail -n 10
sudo systemctl restart wazuh-manager
```

Listing 9.17: Verificação de erros e reinício do gestor Wazuh

10 Análise das Experiências Realizadas

Ao aplicar as políticas de segurança descritas no capítulo 9, é possível observar determinados comportamentos. Esta secção dedica-se ao estudo das experiências realizadas e a análise da comunicação entre os sistemas envolvidos.

10.1 Bloqueio de um IP malicioso

A ligação HTTP pode ser testada usando o comando *curl* no atacante para receber o conteúdo HTML da página Web do agente [32] (**10.1**):

```
curl http://192.168.1.2
curl -I http://192.168.1.2
```

Listing 10.1: Teste HTTP com curl

Também pode ser usado o protocolo telnet [33] com o porto 80 e posteriormente fazer um pedido HTTP válido [32] (para a regra ser despoletada no Wazuh é necessário que o pedido seja válido para gerar um log com a estrutura esperada):

```
telnet 192.168.1.2 80
GET / HTTP/1.1
Host: 192.168.1.2
```

Listing 10.2: Teste HTTP através de Telnet

Depois do primeiro pedido HTTP [32] é criada uma regra na firewall do agente durante 3 minutos que descarta pacotes IP [30] da origem 192.168.1.4 (atacante kali) (**Figuta14**).

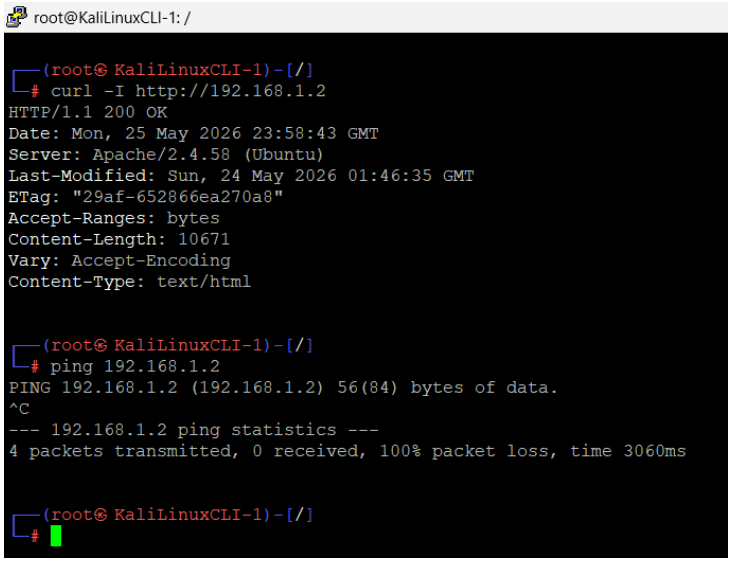


Figura 14: Pedido HTTP e subsequente tentativa de conectividade que falha

10.1.1 Análise de logs

Após a execução da experiência, a análise dos logs gerados no *endpoint* protegido demonstra a eficácia do sistema IPS perante a deteção de um IP catalogado como malicioso.

Os registos no agente do ficheiro `/var/ossec/logs/active-responses.log` evidenciam a execução do script de bloqueio (**10.3**).

```
2026/05/25 23:58:43 active-response/bin/firewall-drop: Starting
2026/05/25 23:58:43 active-response/bin/firewall-drop: {"version":1,"origin
    ":{"name":"node01","module":"wazuh-execd"},"command":"add","parameters":{"
    extra_args":[],"alert":{"timestamp":"2026-05-25T23:58:43.421+0000","rule
    ":{"level":10,"description":"Acesso Web bloqueado de um IP malicioso
    conhecido: 192.168.1.4.","id":"100100","firedtimes":2,"mail":false,"groups
    ":["local"," ip_reputation"]},"agent":{"id":"001","name":"xubuntu-minimal
    ","ip":"192.168.1.2"},"manager":{"name":"wazuh-server"},"id
    ":"1779753523.75970","full_log":"192.168.1.4 - - [25/May/2026:23:58:43
    +0000] \"HEAD / HTTP/1.1\" 200 255 \"-\" \"curl/8.18.0\"","decoder":{"name
    ":"web-accesslog"},"data":{"protocol":"HEAD","srcip":"192.168.1.4","id
    ":"200","url":"/"},"location":"/var/log/apache2/access.log"},"program":"
    active-response/bin/firewall-drop"}}
2026/05/25 23:58:43 active-response/bin/firewall-drop: Ended
```

Listing 10.3: Logs da resposta ativa no agente

Simultaneamente, o servidor Wazuh recebeu e registou a causa deste bloqueio, identificando o pedido HTTP validado pelo servidor Apache [32] e a respetiva ordem de bloqueio (*active-response*):

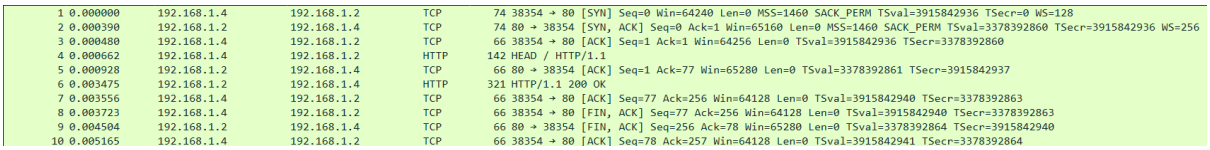
```
2026 May 25 23:58:43 (xubuntu-minimal) any->/var/log/apache2/access.log
   192.168.1.4 - - [25/May/2026:23:58:43 +0000] "HEAD / HTTP/1.1" 200 255 "-"
    "curl/8.18.0"
2026 May 25 23:58:43 (xubuntu-minimal) any->/var/ossec/logs/active-responses.
   log 2026/05/25 23:58:43 active-response/bin/firewall-drop: Starting
2026 May 25 23:58:43 (xubuntu-minimal) any->/var/ossec/logs/active-responses.
   log2026/05/25 23:58:43 active-response/bin/firewall-drop: {"version":1,"
   origin":{"name":"node01","module":"wazuh-execd"},"command":"continue","
   parameters":{"extra_args":[],"alert":{"timestamp":"2026-05-25T23
   :58:43.421+0000","rule":{"level":10,"description":"Acesso Web bloqueado de
    um IP malicioso conhecido:192.168.1.4.","id":"100100","firedtimes":2,"
   mail":false,"groups":["local"," ip_reputation"]},"agent":{"id":"001","name
   ":"xubuntu-minimal","ip":"192.168.1.2"},"manager":{"name":"wazuh-server
   "},"id":"1779753523.75970","full_log":"192.168.1.4 - - [25/May
   /2026:23:58:43 +0000] \"HEAD / HTTP/1.1\" 200 255 \"-\"\"curl/8.18.0\"","
   decoder":{"name":"web-accesslog"},"data":{"protocol":"HEAD","srcip
   ":"192.168.1.4","id":"200","url":"/"},"location":"/var/log/apache2/access.
   log"},"program":"active-response/bin/firewall-drop"}}
2026 May 25 23:58:43 (xubuntu-minimal) any->/var/ossec/logs/active-responses.
   log
2026 May 25 23:58:43 (xubuntu-minimal) any->/var/ossec/logs/active-responses.
   log 2026/05/25 23:58:43 active-response/bin/firewall-drop: Ended
```

Listing 10.4: Logs recebidos no Wazuh Manager

Estes registos demonstram que, embora a arquitetura seja reativa, a resposta é praticamente instantânea. O acesso HTTP foi intercetado e a regra `100100` ativou o bloqueio ao nível da rede [32] imediatamente após a geração do primeiro log.

10.1.2 Análise de pacotes no Wireshark

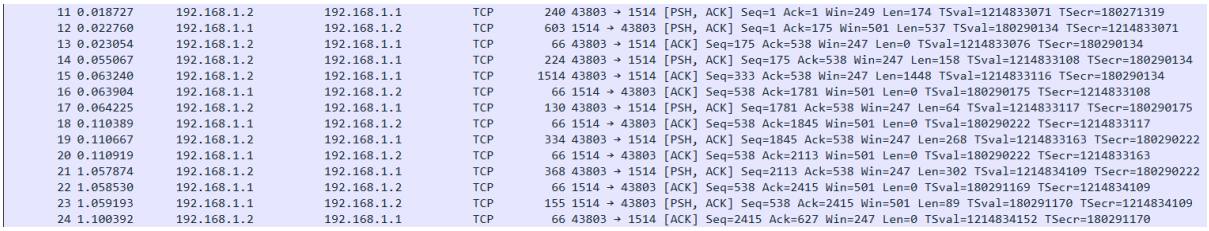
A captura de tráfego da rede confirma o comportamento registado nos logs. Numa fase inicial, é possível observar no Wireshark o estabelecimento da ligação TCP [22] (*Three-way handshake*, SYN -> SYN,ACK -> ACK), a subsequente transferência HTTP bem-sucedida [32] (código `200 OK`) e o término da ligação TCP (FIN,ACK) (Figura 15).



1	0.000000	192.168.1.4	192.168.1.2	TCP	74 38354 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=3915842936 TSecr=0 WS=128
2	0.000390	192.168.1.2	192.168.1.4	TCP	74 80 → 38354 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SACK_PERM TSval=3378392860 TSecr=3915842936 WS=256
3	0.000480	192.168.1.4	192.168.1.2	TCP	66 38354 → 80 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=3915842936 TSecr=3378392860
4	0.000662	192.168.1.4	192.168.1.2	HTTP	142 HEAD / HTTP/1.1
5	0.000928	192.168.1.2	192.168.1.4	TCP	66 80 → 38354 [ACK] Seq=1 Ack=77 Win=65280 Len=0 TSval=3378392861 TSecr=3915842937
6	0.003475	192.168.1.2	192.168.1.4	HTTP	321 HTTP/1.1 200 OK
7	0.003556	192.168.1.4	192.168.1.2	TCP	66 38354 → 80 [ACK] Seq=77 Ack=256 Win=64128 Len=0 TSval=3915842940 TSecr=3378392863
8	0.003723	192.168.1.4	192.168.1.2	TCP	66 38354 → 80 [FIN, ACK] Seq=77 Ack=256 Win=64128 Len=0 TSval=3915842940 TSecr=3378392863
9	0.004504	192.168.1.2	192.168.1.4	TCP	66 80 → 38354 [FIN, ACK] Seq=256 Ack=78 Win=65280 Len=0 TSval=3378392864 TSecr=3915842940
10	0.005165	192.168.1.4	192.168.1.2	TCP	66 38354 → 80 [ACK] Seq=78 Ack=257 Win=64128 Len=0 TSval=3915842941 TSecr=3378392864

Figura 15: Ligação HTTP com pedido e resposta efetuados

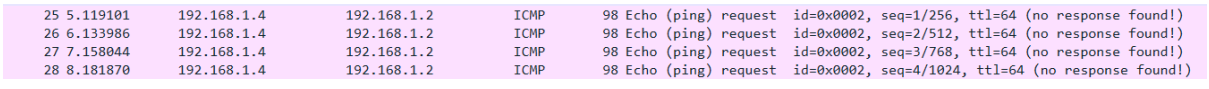
Após a ligação TCP no porto 80, é possível observar outra ligação TCP que usa o porto 1514 do Wazuh. Os PDUs desta ligação correspondem ás trocas protocolares entre o servidor Wazuh (192.168.1.1) e o seu agente (192.168.1.2), alguns destes PDUs têm a flag PSH ativada (os dados são enviados imediatamente para a aplicação do Wazuh em vez de serem colocados numa *cache*) e contêm dados cifrados o que indica que se tratam de PDUs que contêm logs. As outras PDUs desta ligação TCP têm apenas a flag ACK ativada e não transportam dados dentro do segmento TCP (indicando que servem simplesmente para cumprir os requisitos do protocolo TCP) (Figura 16).



11	0.018727	192.168.1.2	192.168.1.1	TCP	240 43803 → 1514 [PSH, ACK] Seq=1 Ack=1 Win=249 Len=174 TSval=1214833071 TSecr=180271319
12	0.022760	192.168.1.1	192.168.1.2	TCP	603 1514 → 43803 [PSH, ACK] Seq=1 Ack=175 Win=501 Len=537 TSval=180290134 TSecr=1214833071
13	0.023054	192.168.1.2	192.168.1.1	TCP	66 43803 → 1514 [ACK] Seq=175 Ack=538 Win=247 Len=0 TSval=1214833076 TSecr=180290134
14	0.055067	192.168.1.2	192.168.1.1	TCP	224 43803 → 1514 [PSH, ACK] Seq=175 Ack=538 Win=247 Len=158 TSval=1214833108 TSecr=180290134
15	0.063240	192.168.1.2	192.168.1.1	TCP	1514 43803 → 1514 [ACK] Seq=333 Ack=538 Win=247 Len=1448 TSval=1214833116 TSecr=180290134
16	0.063904	192.168.1.1	192.168.1.2	TCP	66 1514 → 43803 [ACK] Seq=538 Ack=1781 Win=501 Len=0 TSval=180290175 TSecr=1214833108
17	0.064225	192.168.1.2	192.168.1.1	TCP	130 43803 → 1514 [PSH, ACK] Seq=1781 Ack=538 Win=247 Len=64 TSval=1214833117 TSecr=180290175
18	0.110389	192.168.1.1	192.168.1.2	TCP	66 1514 → 43803 [ACK] Seq=538 Ack=1845 Win=501 Len=0 TSval=180290222 TSecr=1214833117
19	0.110667	192.168.1.2	192.168.1.1	TCP	334 43803 → 1514 [PSH, ACK] Seq=1845 Ack=538 Win=247 Len=268 TSval=1214833163 TSecr=180290222
20	0.110919	192.168.1.1	192.168.1.2	TCP	66 1514 → 43803 [ACK] Seq=538 Ack=2113 Win=501 Len=0 TSval=180290222 TSecr=1214833163
21	1.057874	192.168.1.2	192.168.1.1	TCP	368 43803 → 1514 [PSH, ACK] Seq=2113 Ack=538 Win=247 Len=302 TSval=1214834109 TSecr=180290222
22	1.058530	192.168.1.1	192.168.1.2	TCP	66 1514 → 43803 [ACK] Seq=538 Ack=2415 Win=501 Len=0 TSval=180291169 TSecr=1214834109
23	1.059193	192.168.1.1	192.168.1.2	TCP	155 1514 → 43803 [PSH, ACK] Seq=538 Ack=2415 Win=501 Len=89 TSval=180291170 TSecr=1214834109
24	1.100392	192.168.1.2	192.168.1.1	TCP	66 43803 → 1514 [ACK] Seq=2415 Ack=627 Win=247 Len=0 TSval=1214834152 TSecr=180291170

Figura 16: Ligação TCP entre servidor Wazuh e agente

Depois disto foram capturados 4 pacotes ICMP [35] da tentativa de conectividade que falhou (o Wireshark indica: (*no response found!*)). Sendo a regra ativada da firewall baseada numa política de *DROP* e não de *REJECT*, o agente descarta os pacotes silenciosamente no seu *iptables* (Figura 17).



25	5.119101	192.168.1.4	192.168.1.2	ICMP	98 Echo (ping) request id=0x0002, seq=1/256, ttl=64 (no response found!)
26	6.133986	192.168.1.4	192.168.1.2	ICMP	98 Echo (ping) request id=0x0002, seq=2/512, ttl=64 (no response found!)
27	7.158044	192.168.1.4	192.168.1.2	ICMP	98 Echo (ping) request id=0x0002, seq=3/768, ttl=64 (no response found!)
28	8.181870	192.168.1.4	192.168.1.2	ICMP	98 Echo (ping) request id=0x0002, seq=4/1024, ttl=64 (no response found!)

Figura 17: Pacotes ICMP echo request sem resposta

10.2 Bloqueio dinâmico (SSH e Telnet)

Para testar a política de segurança contra ataques de força bruta, simulou-se um ataque ao serviço SSH [31] do agente Xubuntu a partir da máquina Kali Linux.

De forma a automatizar o ataque de forma simples, utilizou-se a ferramenta *Hydra* no terminal do atacante, de forma a executar um ataque de força bruta e testar a regra aplicada no sevidor Wazuh(**10.5**).

```
hydra -l wazuh -x 1:1:abcdefghij ssh://192.168.1.2
```

Listing 10.5: Execução do ataque de força bruta com Hydra

O comando vai testar passwords de 1 carácter e resulta em 35 tentativas de *login* o que é suficiente para ativar a resposta ativa no servidor Wazuh. A ferramenta é agressiva (força bruta) logo testa passwords extra e abre várias ligações SSH simultâneas[31].

Imediatamente depois do ataque efetuou-se um teste de conectividade IP (ICMP ping) a partir do atacante:

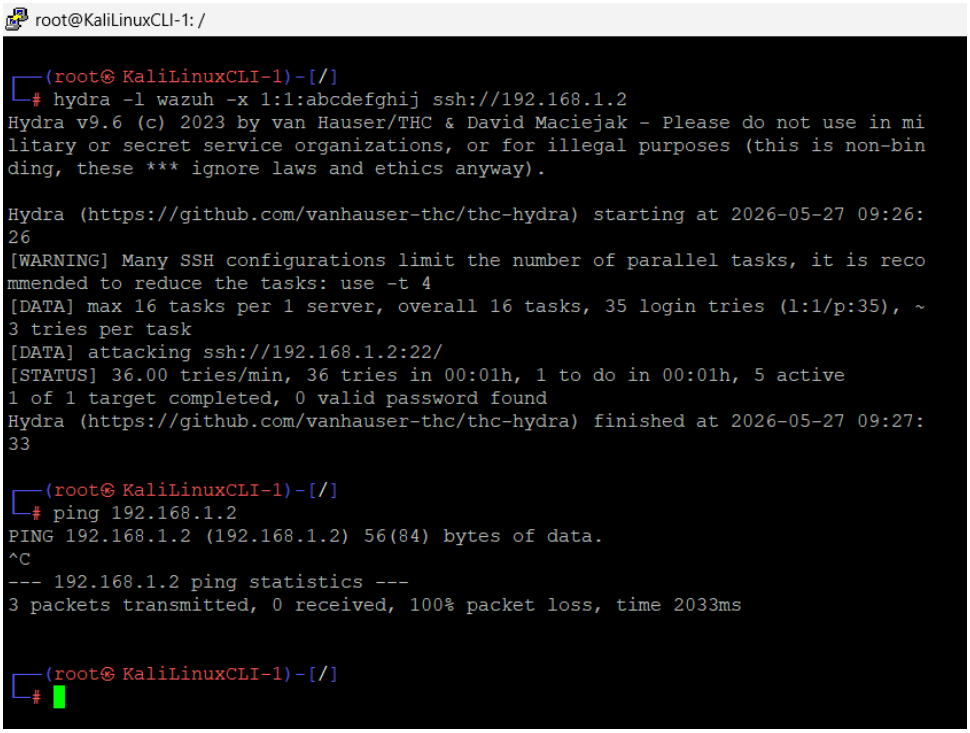


Figura 18: Teste de força bruta e subsequente tentativa de conectividade que falha

O resultado foi a ausência total de uma resposta ICMP, evidenciando o bloqueio da origem do ataque ao nível da rede.

10.2.1 Análise de logs

A análise dos logs neste caso comprova a deteção e o desencadeamento da regra configurada contra ataques de força bruta. No agente foi possível detetar logs do script a ser iniciado. (**10.6**).

```
2026/05/27 09:26:29 active-response/bin/firewall-drop: Starting
2026/05/27 09:26:29 active-response/bin/firewall-drop: {"version":1,"origin
    ":{"name":"node01","module":"wazuh-execd"},"command":"add","parameters":{"
    extra_args":[],"alert":{"timestamp":"2026-05-27T09:26:29.294+0000","rule
    ":{"level":10,"description":"PAM: Multiple failed logins in a small period
     of time.","id":"5551","mitre":{"id":["T1110"],"tactic":["Credential
    Access"],"technique":["Brute Force"]},"frequency":8,"firedtimes":1,"mail":
    false,"groups":["pam","syslog","authentication_failures"],"pci_dss
    ":["10.2.4","10.2.5","11.4"],"gpg13":["7.8"],"gdpr":["IV_35.7.d","IV_32
    .2"],"hipaa":["164.312.b"],"nist_800_53":["AU.14","AC.7","SI.4"],    "tsc
    ":["CC6.1","CC6.8","CC7.2","CC7.3"]},"agent":{"id":"001","name":"xubuntu-
    minimal","ip":"192.168.1.2"},"manager":{"name":"wazuh-server"},"id
    ":"1779873989.132518","previous_output":"May 27 09:26:27 wazuh-agent sshd
    [3614]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0
     tty=ssh ruser= rhost=192.168.1.4  user=wazuh\nMay 27 09:26:27 wazuh-agent
     sshd[3605]: pam_unix(sshd:auth): authentication failure; logname= uid=0
    euid=0 tty=ssh ruser= rhost=192.168.1.4  user=wazuh\nMay 27 09:26:27 wazuh
    -agent sshd[3612]: pam_unix(sshd:auth): authentication failure; logname=
    uid=0 euid=0 tty=ssh ruser= rhost=192.168.1.4 user=wazuh\nMay 27 09:26:27
    wazuh-agent sshd[3609]: pam_unix(sshd:auth): authentication failure;
    logname= uid=0 euid=0 tty=ssh ruser= rhost=192.168.1.4  user=wazuh\nMay 27
     09:26:27 wazuh-agent sshd[3607]: pam_unix(sshd:auth): authentication
    failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.168.1.4  user=
    wazuh\nMay 27 09:26:27 wazuh-agent sshd[3611]: pam_unix(sshd:auth):
    authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost
    =192.168.1.4  user=wazuh\nMay 27 09:26:27 wazuh-agent sshd[3603]: pam_unix
    (sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser=
    rhost=192.168.1.4  user=wazuh","full_log":"May 27 09:26:27 wazuh-agent
    sshd[3619]: pam_unix(sshd:auth): authentication failure; logname= uid=0
    euid=0 tty=ssh ruser= rhost=192.168.1.4  user=wazuh","predecoder":{"
    program_name":"sshd","timestamp":"May 27 09:26:27","hostname":"wazuh-agent
    "},"decoder":{"name":"pam"},"data":{"srcip":"192.168.1.4","dstuser":"wazuh
    ","uid":"0","euid":"0","tty":"ssh"},"location":"journald"},"program":"
    active-response/bin/firewall-drop"}}
2026/05/27 09:26:29 active-response/bin/firewall-drop: Ended
```

Listing 10.6: Logs da resposta ativa após força bruta

No servidor Wazuh foi possível observar vários logs de erros de autenticação e de passwords erradas de vários processos SSH a correr no agente [31] (a ferramenta Hydra estabelece várias sessões SSH ao mesmo tempo) e logs de erro identificados como *MaxStartups throttling* indicando um abrandamento da quantidade de ligações SSH no agente. Estes logs pertencem ao serviço SSH e fazem da parte da sua funcionalidade base. Foram omitidos alguns destes logs para efeitos de clareza. Depois dos logs do SSH é possível observar os logs do *active-response* e a respetiva ativação do script *firewall-drop*. (**10.7**).

```
2026 May 27 09:25:59 (xubuntu-minimal) any->journald May 27 09:25:58 wazuh-
    agent sudo[3598]: pam_unix(sudo:session): session opened for user root(uid
    =0) by wazuh(uid=1000)
2026 May 27 09:26:27 (xubuntu-minimal) any->journald May 27 09:26:27 wazuh-
    agent sshd[3601]: Received disconnect from 192.168.1.4 port 57952:11: Bye
    Bye [preauth]
2026 May 27 09:26:27 (xubuntu-minimal) any->journald May 27 09:26:27 wazuh-
    agent sshd[3601]: Disconnected from authenticating user wazuh 192.168.1.4
    port 57952 [preauth]
2026 May 27 09:26:29 (xubuntu-minimal) any->journald May 27 09:26:27 wazuh-
    agent sshd[1406]: error: beginning MaxStartups throttling
2026 May 27 09:26:29 (xubuntu-minimal) any->journald May 27 09:26:27 wazuh-
    agent sshd[1406]: drop connection #13 from [192.168.1.4]:58072 on
    [192.168.1.2]:22 past MaxStartups
2026 May 27 09:26:29 (xubuntu-minimal) any->journald May 27 09:26:27 wazuh-
    agent sshd[3603]: pam_unix(sshd:auth): authentication failure; logname=
    uid=0 euid=0 tty=ssh ruser= rhost=192.168.1.4 user=wazuh
... (logs idênticos ao de cima omitidos) ...
2026 May 27 09:26:29 (xubuntu-minimal) any->journald May 27 09:26:29 wazuh-
    agent sshd[3603]: Failed password for wazuh from 192.168.1.4 port 57958
    ssh2
... (logs idênticos ao de cima omitidos) ...
2026 May 27 09:26:30 (xubuntu-minimal) any->/var/ossec/logs/active-responses.
    log 2026/05/27 09:26:29 active-response/bin/firewall-drop: Starting
2026 May 27 09:26:30 (xubuntu-minimal) any->/var/ossec/logs/active-responses.
    log 2026/05/27 09:26:29 active-response/bin/firewall-drop: {"version":1,"
    origin":{"name":"node01","module":"wazuh-execd"},"command":"continue","
    parameters":{"extra_args":[],"alert":{"timestamp":"2026-05-27T09
    :26:29.294+0000","rule":{"level":10,"description":"PAM: Multiple failed
    logins in a small period of time.","id":"5551","mitre":{"id":["T1110"],"
    tactic":["Credential Access"],"technique":["Brute Force"]},"frequency":8,"
    firedtimes":1,"mail":false,"groups":["pam","syslog","
    authentication_failures"],"pci_dss":["10.2.4","10.2.5","11.4"],"gpg13
    ":["7.8"],"gdpr":["IV_35.7.d","IV_32.2"],"hipaa":["164.312.b"],"
```

```
    nist_800_53":["AU.14","AC.7","SI.4"],"tsc":["CC6.1","CC6.8","CC7.2","CC7
    .3"]},"agent":{"id":"001","name":"xubuntu-minimal","ip":"192.168.1.2"},"
    manager":{"name":"wazuh-server"},"id":"1779873989.132518","previous_output
    ":"May 27 09:26:27 wazuh-agent sshd[3614]: pam_unix(sshd:auth):
    authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost
    =192.168.1.4  user=wazuh\nMay 27 09:26:27 wazuh-agent sshd[3611]: pam_unix
    (sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser=
    rhost=192.168.1.4  user=wazuh\nMay 27 09:26:27 wazuh-agent sshd[3603]:
    pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh
     ruser= rhost=192.168.1.4  user=wazuh","full_log":"May 27 09:26:27 wazuh-
    agent sshd[3619]: pam_unix(sshd:auth): authentication failure; logname=
    uid=0 euid=0 tty=ssh ruser= rhost=192.168.1.4  user=wazuh","predecoder":{"
    program_name":"sshd","timestamp":"May 27 09:26:27","hostname":"wazuh-agent
    "},"decoder":{"name":"pam"},"data":{"srcip":"192.168.1.4","dstuser":"wazuh
    ","uid":"0","euid":"0","tty":"ssh"},"location":"journald"},"program":"
    active-response/bin/firewall-drop"}}
2026 May 27 09:26:30 (xubuntu-minimal) any->/var/ossec/logs/active-responses.
    log 2026/05/27 09:26:29 active-response/bin/firewall-drop: Ended
2026 May 27 09:29:03 (xubuntu-minimal) any->journald May 27 09:29:02 wazuh-
    agent sudo[3598]: pam_unix(sudo:session): session closed for user root
```

Listing 10.7: Logs de autenticação recebidos no Wazuh Manager

10.2.2 Análise de pacotes no Wireshark

Ao iniciar o ataque é possível observar o estabelecimento inicial de várias ligações SSHv2: pacotes TCP com a flag SYN ativada [22], troca de algoritmos e chaves privadas seguido de pacotes SSHv2 cifrados (Figura 19).

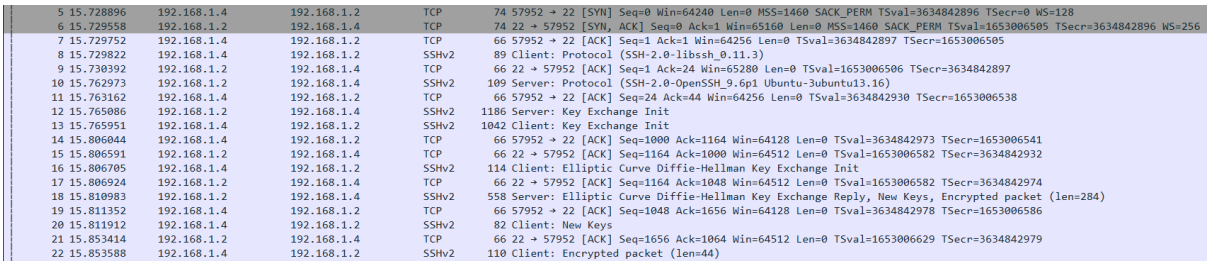


Figura 19: Estabelecimento de uma das sessões SSHv2

Depois de todas as sessões SSH [31] estarem estabelecidas começa-se apenas a observar pacotes TCP [22] catalogados no Wireshark como "TCP Spurious Retransmission", "TCP Dup ACK"e "TCP Retransmission", isto comprova o mecanismo de defesa *active response* a funcionar corretamente com a regra da firewall DROP do agente a bloquear o IP atacante. O Hydra continua a tentar passwords mas com a regra da firewall ativada as diferentes ligações TCP entre o atacante e o agente enviam pacotes duplicados ou com dados já recebidos e confirmados (Figura 20).

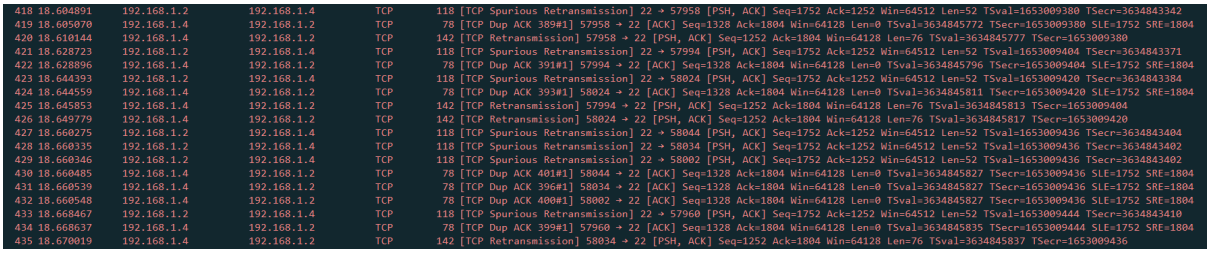


Figura 20: Segmentos TCP que validam o sistema IPS

No final é possível observar 5 pacotes TCP com a flag FIN ativada com origem no atacante indicando que o Hydra utilizou 5 ligações TCP para o ataque e os pacotes ICMP sem resposta do comando *ping* (Figura 21).

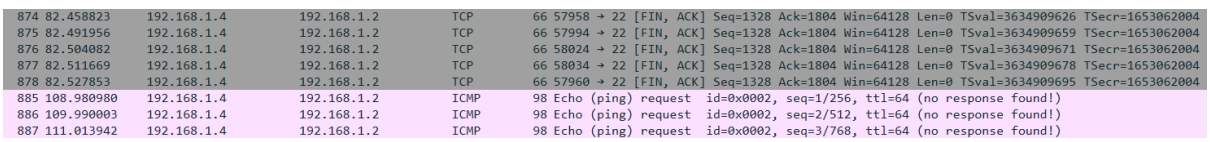


Figura 21: Encerramento das ligações TCP por parte do atacante e teste de conectividade

10.3 Proteção contra *SYN floods* & *Port Scans*

Para testar a eficácia das políticas de segurança implementadas no **capítulo 9**, foram realizados diversos ataques usando as ferramentas Nmap e Hping3(Sec2.2) que permitem testar a proteção implementada contra ataques de *port scans* e *SYN floods*. Dado que o sensor de ataques nesta secção é o router (ou mais especificamente as acls configuradas no router) os pacotes do atacante têm de ter destino nas portas do router ou passar pelo router para que o syslog consiga enviar logs para o servidor Wazuh.

O primeiro ataque testado nesta secção tem origem num atacante que se encontra na rede do servidor Wazuh e do agente e ataca diretamente a porta do router virada para esta rede.

10.3.1 Bloqueio de *port scans*

Um ataque de varrimento de portos serve principalmente para recolher o máximo de informação possível do sistema alvo, o que pode permitir posteriormente um ataque mais adaptado ao sistema que explora um certo serviço ou vulnerabilidade descoberta. Neste caso o atacante executa o comando *nmap* (Figura 22).

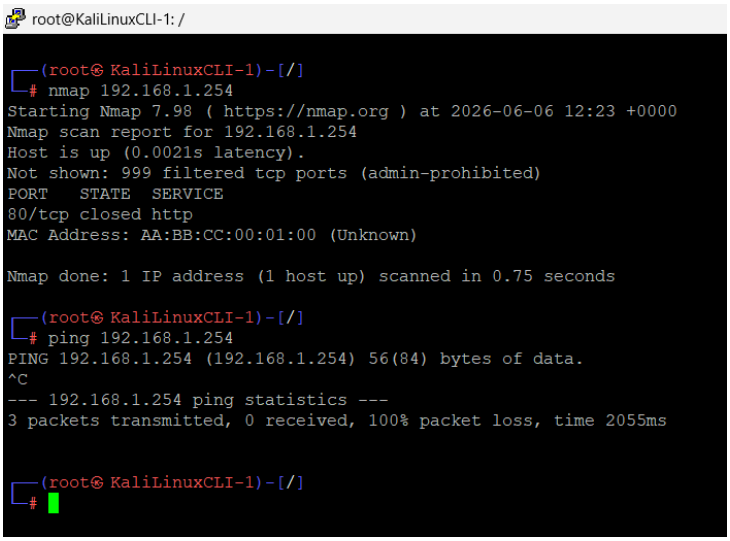


Figura 22: *Port scan* usando *nmap*

Foi depois testada a conectividade ao router usando o *ping*, os pacotes chegam ao router mas ao atingirem a *null route* são enviados para o lixo e não são devolvidos pacotes *echo reply* comprovando a eficácia do sistema IPS, o atacante é basicamente proibido de utilizar os serviços de encaminhamento do router durante 10 minutos.

Os logs relevantes podem ser consultados no router e no Wazuh server, no caso do router, os logs são provenientes das ACL configuradas e do serviço Telnet [33] e indicam pacotes a serem negados e configurações em linhas virtuais do router que corresponde aos comandos a serem introduzidos pelo script. Existem logs a serem gerados no router depois da *null route* ser introduzida no router, isto corresponde a logs atrasados gerados pelo router do ataque que já passou. A tabela de encaminhamento do router foi consultada antes e depois de a *null route* ser introduzida:

```
R1#
00:02:42: %SEC-6-IPACCESSLOGP: list MASTER_POLICY denied tcp
    192.168.1.4(63164) -> 192.168.1.254(5900), 1 packet
R1#
00:02:44: %SEC-6-IPACCESSLOGP: list MASTER_POLICY permitted tcp
    192.168.1.3(56590) -> 34.107.221.82(80), 1 packet
R1#
00:02:47: %SYS-5-CONFIG_I: Configured from console by cisco on vty0
    (192.168.1.1)
R1#
00:03:00: %SEC-6-IPACCESSLOGRL: access-list logging rate-limited or missed
    20952 packets
R1#
00:08:00: %SEC-6-IPACCESSLOGP: list MASTER_POLICY denied tcp
    192.168.1.4(63164) -> 192.168.1.254(5004), 1 packet
00:08:00: %SEC-6-IPACCESSLOGP: list MASTER_POLICY denied tcp
    192.168.1.4(63164) -> 192.168.1.254(880), 1 packet
00:08:00: %SEC-6-IPACCESSLOGP: list MASTER_POLICY denied tcp
    192.168.1.4(63164) -> 192.168.1.254(8045), 1 packet
00:08:00: %SEC-6-IPACCESSLOGP: list MASTER_POLICY denied tcp
    192.168.1.4(63164) -> 192.168.1.254(1061), 1 packet
(... mais linhas como a de cima omitidas)
```

Listing 10.8: Logs visualizados no router

```
(...informacao omitida)
Gateway of last resort is 192.168.42.1 to network 0.0.0.0
S*    0.0.0.0/0 [254/0] via 192.168.42.1
      192.168.1.0/24 is variably subnetted, 3 subnets, 2 masks
C        192.168.1.0/24 is directly connected, Ethernet0/0
S        192.168.1.4/32 is directly connected, Null0
L        192.168.1.254/32 is directly connected, Ethernet0/0
      192.168.2.0/24 is variably subnetted, 2 subnets, 2 masks
C        192.168.2.0/24 is directly connected, Ethernet0/2
L        192.168.2.254/32 is directly connected, Ethernet0/2
      192.168.42.0/24 is variably subnetted, 2 subnets, 2 masks
C        192.168.42.0/24 is directly connected, Ethernet0/1
L        192.168.42.134/32 is directly connected, Ethernet0/1
```

Listing 10.9: Tabela de encaminhamento pós ação do IPS

```
(...informacao omitida)
S*    0.0.0.0/0 [254/0] via 192.168.42.1
      192.168.1.0/24 is variably subnetted, 2 subnets, 2 masks
C        192.168.1.0/24 is directly connected, Ethernet0/0
L        192.168.1.254/32 is directly connected, Ethernet0/0
      192.168.2.0/24 is variably subnetted, 2 subnets, 2 masks
C        192.168.2.0/24 is directly connected, Ethernet0/2
L        192.168.2.254/32 is directly connected, Ethernet0/2
      192.168.42.0/24 is variably subnetted, 2 subnets, 2 masks
C        192.168.42.0/24 is directly connected, Ethernet0/1
L        192.168.42.134/32 is directly connected, Ethernet0/1
R1#
```

Listing 10.10: Tabela de encaminhamento após termino da regra

No caso do Wazuh server podem ser analisados os logs registados no ficheiro do *active response* que é onde o script descreve as suas ações. É possivel observar o script IPS a ser ativado com o comand "ADD"e as informações do comando a serem aplicadas no router, depois disso são observados logs que indicam que o Wazuh está a receber logs do router mas já tomou ação sobre o ataque, estes logs dão reset ao timer de 10 minutos o que faz com que o timeout se prolongue, o comando que retira a *null route* é enviado 10 minutos depois usando comand "DELETE"no script Python.

```
2026/06/06 12:23:29 active-response/bin/cisco_ips.py: Custom IPS iniciado ->
    Comando: ADD | Regra: 100301
2026/06/06 12:23:29 active-response/bin/cisco_ips.py: A injetar null route no
    router. IP alvo: 192.168.1.4
2026/06/06 12:23:32 active-response/bin/cisco_ips.py: SUCESSO: Comandos
    aplicados no router (ADD)!
2026/06/06 12:28:45 active-response/bin/cisco_ips.py: Info: Acao repetida
    ignorada. A ameaca ja se encontra bloqueada pelo Wazuh.
2026/06/06 12:28:45 active-response/bin/cisco_ips.py: Info: Acao repetida
    ignorada. A ameaca ja se encontra bloqueada pelo Wazuh.
2026/06/06 12:28:47 active-response/bin/cisco_ips.py: Info: Acao repetida
    ignorada. A ameaca ja se encontra bloqueada pelo Wazuh.
2026/06/06 12:28:47 active-response/bin/cisco_ips.py: Info: Acao repetida
    ignorada. A ameaca ja se encontra bloqueada pelo Wazuh.
(... mais logs como os de cima omitidos)
2026/06/06 12:38:48 active-response/bin/cisco_ips.py: Custom IPS iniciado ->
    Comando: DELETE | Regra: 100301
2026/06/06 12:38:48 active-response/bin/cisco_ips.py: Timeout (Stateful). A
    remover null route para: 192.168.1.4
2026/06/06 12:38:51 active-response/bin/cisco_ips.py: SUCESSO: Comandos
    aplicados no router (DELETE)!
```

Listing 10.11: Logs visualizados no Wazuh provenientes do script IPS

No Wireshark é possível observar muitos pacotes ICMP a serem *administratively filtered* (comando nmap a ser bloqueado e router a avisar fonte do ataque) e pacotes TCP sem dados que corresponde ás tentativas de scan do comando nmap (Figura 23).

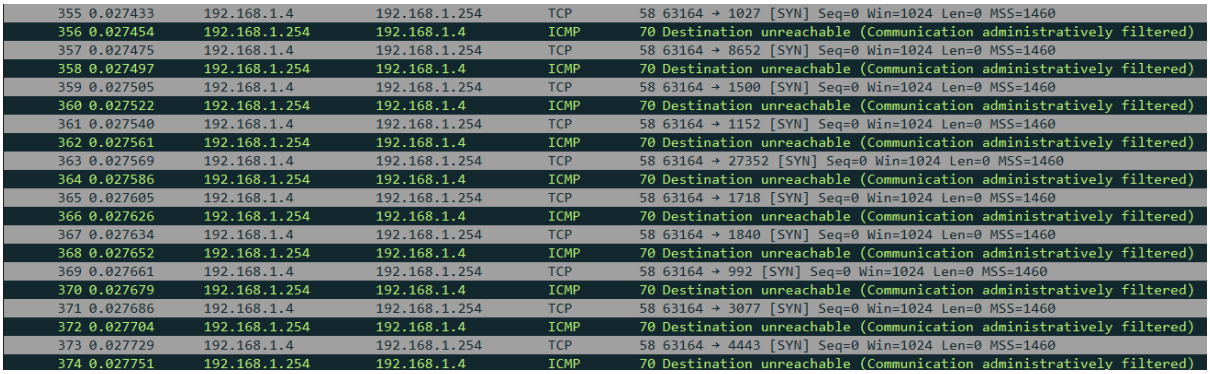


Figura 23: Pacotes TCP do *nmap* e avisos ICMP *filtered*

Depois de tantos pacotes enviados para a rede é finalmente enviado um pacote Syslog pelo router a indicar ao Wazuh que a ACL bloqueou um pacote TCP e posteriormente é possível observar pacotes TCP e TELNET correspondentes á ligação do script Python a enviar o comando ao router (Figura 24).

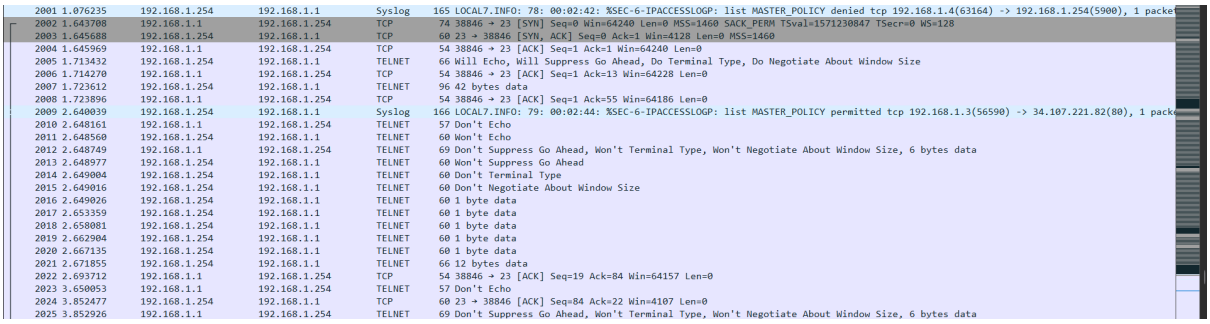


Figura 24: Primeiro pacote Syslog e ligação Telnet enviado

Depois desta ligação Telnet [33] é possível observar outros logs enviados pelo router para o Wazuh (log da configuração pela linha virtual e logging rate-limited) e a tentativa de conectividade pelo atacante no meio desses pacotes (Figura 25).

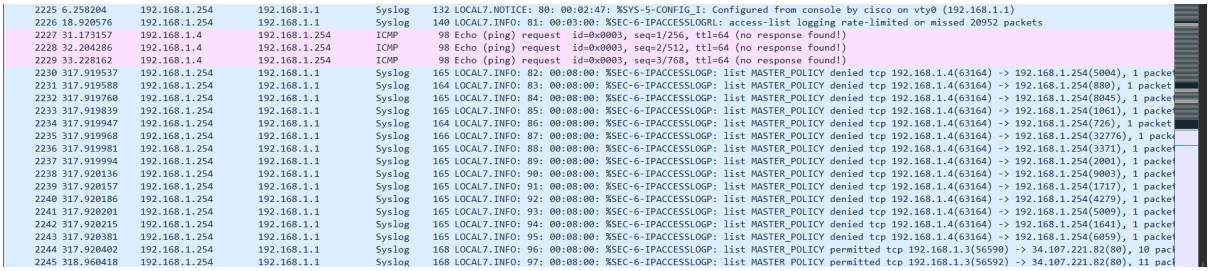


Figura 25: Pacotes Syslog provenientes do router e teste do *ping* no atacante

Por fim é estabelecida outra ligação Telnet [33] que corresponde á segunda chamada do script que retira a *null route* do router e depois o pacote Syslog que identifica a ligação á linha virtual (Figura 26).

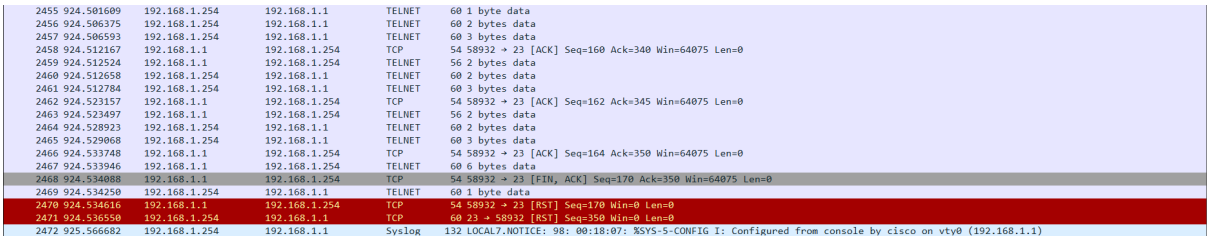


Figura 26: Alguns pacotes da segunda ligação Telnet e o pacote Syslog final da experiência

Devido à ACL configurada no router, que protege o agente, é possível iniciar um ataque de *port scan* com origem numa rede distinta da rede do agente, passando pelo router e tendo como destino o agente Xubuntu. Neste cenário, o atacante também é penalizado (Figura 27).

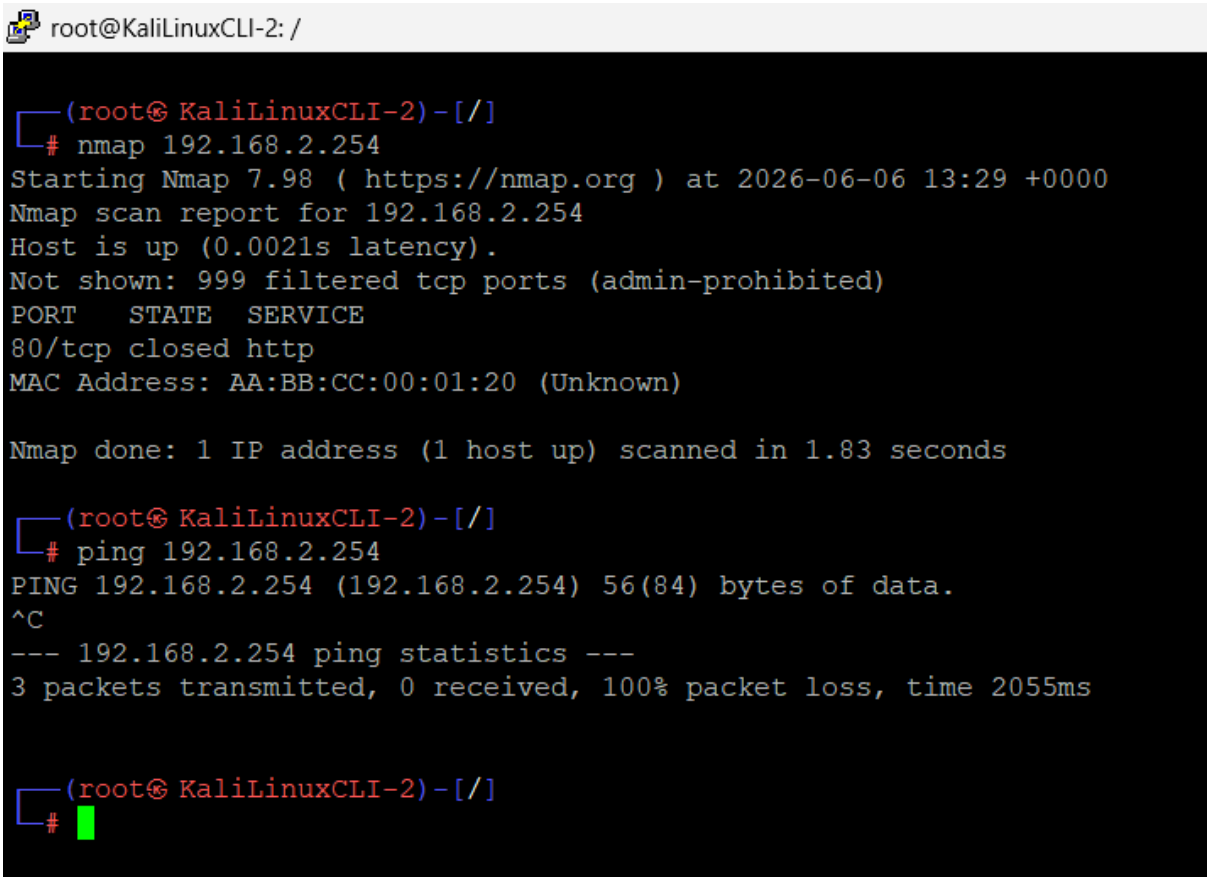


Figura 27: Ataque ao agente Xubuntu e posterior teste de conectividade que falha no atacante

No Wireshark, é possível observar uma sequência de pacotes ICMP gerados pela ACL do router, bem como os pacotes TCP enviados pelo atacante. No final da captura, observa-se ainda o teste de conectividade realizado com o comando *ping* (Figura 28).

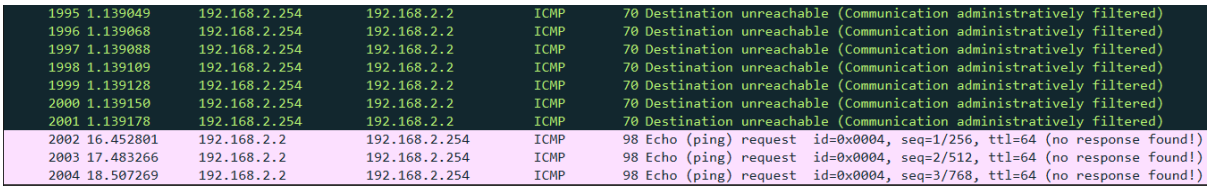


Figura 28: Pacotes filtrados e pacotes ICMP sem resposta

Neste caso os logs gerados no Wazuh server e no router são idênticos ao da experiência anterior. A routa estática introduzida no router é agora da rede desprotegida (por exemplo o atacante com o IP 192.168.2.2 a atacar a porta do router com o IP 192.168.2.254). A maior diferença é que o ataque fica restrito á rede do atacante dado que os pacotes são imediatamente bloqueados no router através da ACL. O agente fica sem qualquer conhecimento que o ataque aconteceu porque são trocados pacotes apenas entre o Wazuh server e o router e entre o router e o atacante.

10.3.2 Bloqueio de *SYN floods*

Nesta experiência o router é protegido contra pacotes TCP [22] fraudulentos gerados com o comando *hping3* [14], que tem a capacidade de enviar uma enorme quantia de pacotes TCP com IPs de origem forjados (spoofed) usando o porto 80 de forma contínua. Usando comandos que limitam a taxa de pacotes TCP que passam pelo router, sistemas de destino que tenham o router como intermédio acabam por ser protegidos também.

Neste caso é testado um ataque com origem num atacante na rede 192.168.2.0/24 com destino no agente Xubuntu com o IP 192.168.1.2. Como o enorme fluxo de pacotes TCP passam pelo router, a resposta ativa no Wazuh é ativada.

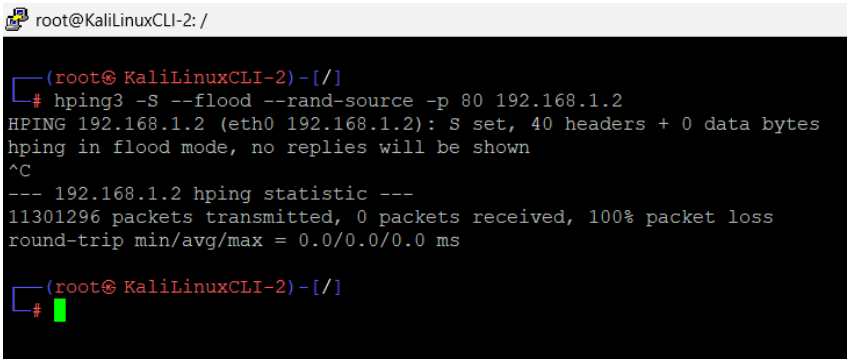


Figura 29: Ataque *SYN flood* efetuado no atacante Kali-Linux

Os pacotes enviados pelo atacante Kali-Linux chegam à interface do router 192.168.2.2 e consistem em segmentos TCP sem dados, com a flag SYN ativada e com destino ao agente Xubuntu (192.168.1.2) e com IPs de origem *spoofed*. Foram capturados cerca de 400000 pacotes maliciosos nesta experiência que servem puramente para causar um ataque DoS no router.

Nos logs do router é possível observar a ACL MASTER_POLICY a permitir muitos pacotes de origens falsas. Assim que o router é configurado com a política de segurança para abrandar os pacotes TCPs (*Configured from console by cisco on vty0*) é gerada mais ou menos no mesmo instante um log *TCP-6-INTERCEPT* que indica a sua ativação. Depois disso o router começa a permitir muito menos pacotes e momentos depois de cancelar o comando *hping3* é gerado o log *%TCP-6-INTERCEPT: calming down* que indica que a taxa de receção de pacotes TCP está a acalmar no router. Por fim o script Python restaura o router para o estado anterior desativando o TCP intercept (*00:22:05: %SYS-5-CONFIG_I: Configured from console by cisco on vty0*).

```
R1#
00:05:52: %SEC-6-IPACCESSLOGP: list MASTER_POLICY permitted tcp
    34.213.43.183(2427) -> 192.168.1.2(80), 1 packet
00:05:53: %SEC-6-IPACCESSLOGP: list MASTER_POLICY permitted tcp
    145.119.179.118(20705) -> 192.168.1.2(80), 1 packet
R1#
00:05:53: %AMDP2_FE-6-EXCESSCOLL: Ethernet0/0 TDR=0, TRC=0
00:05:54: %SEC-6-IPACCESSLOGP: list MASTER_POLICY permitted tcp
    92.113.224.37(29028) -> 192.168.1.2(80), 1 packet
R1#
(... OMITIDOS...)
00:06:12: %SEC-6-IPACCESSLOGP: list MASTER_POLICY permitted tcp
    76.91.168.141(45324)
00:06:18: %SEC-6-IPACCESSLOGP: list MASTER_POLICY permitted tcp
    145.122.166.152(34690) -> 192.168.1.2(80), 1 packet
00:06:19: %TCP-6-INTERCEPT: getting aggressive, count (500/500) 1 min 0
00:06:19: %SEC-6-IPACCESSLOGP: list MASTER_POLICY permitted tcp
    188.79.100.53(15975) -> 192.168.1.2(80), 1 packet
00:06:20: %SYS-5-CONFIG_I: Configured from console by cisco on vty0
    (192.168.1.1)
R1#
(... OMITIDOS...)
00:06:24: %SEC-6-IPACCESSLOGP: list MASTER_POLICY permitted tcp
    36.104.105.206(40470) -> 192.168.1.2(80), 1 packet
00:06:25: %AMDP2_FE-6-EXCESSCOLL: Ethernet0/2 TDR=0, TRC=0
00:06:25: %SEC-6-IPACCESSLOGP: list MASTER_POLICY permitted tcp
    226.220.189.6(33638) -> 192.168.1.2(80), 1 packet
R1#
00:06:26: %SEC-6-IPACCESSLOGP: list MASTER_POLICY permitted tcp
    52.117.123.111(18990) -> 192.168.1.2(80), 1 packet
(... OMITIDOS...)
00:06:36: %SEC-6-IPACCESSLOGP: list MASTER_POLICY permitted tcp
    170.75.119.236(58200) -> 192.168.1.2(80), 1 packet
00:06:54: %TCP-6-INTERCEPT: calming down, count (49/50) 1 min 0
R1#
00:07:01: %SEC-6-IPACCESSLOGRL: access-list logging rate-limited or missed
    352233 packets
(... OMITIDOS...)
00:22:05: %SYS-5-CONFIG_I: Configured from console by cisco on vty0
    (192.168.1.1)
```

Listing 10.12: Syslog referente ao ataque *SYN flood* no router

Nos logs do *active response* no Wazuh server é possível identificar o início da execução do script IPS, várias tentativas de execução do script mas que foram ignoradas porque a regra já tinha sido despoletada e finalmente o fim da resposta ativa.

```
2026/06/06 15:09:03 active-response/bin/cisco_ips.py: Custom IPS iniciado ->
    Comando: ADD | Regra: 100302
2026/06/06 15:09:03 active-response/bin/cisco_ips.py: A injetar protecao TCP
    Intercept no router (anti SYN Flood).
2026/06/06 15:09:20 active-response/bin/cisco_ips.py: SUCESSO: Comandos
    aplicados no router (ADD)!
2026/06/06 15:09:27 active-response/bin/cisco_ips.py: Info: Acao repetida
    ignorada. A ameaca ja se encontra bloqueada pelo Wazuh.
2026/06/06 15:09:39 active-response/bin/cisco_ips.py: Info: Acao repetida
    ignorada. A ameaca ja se encontra bloqueada pelo Wazuh.
2026/06/06 15:09:39 active-response/bin/cisco_ips.py: Info: Acao repetida
    ignorada. A ameaca ja se encontra bloqueada pelo Wazuh.
2026/06/06 15:15:00 active-response/bin/cisco_ips.py: Info: Acao repetida
    ignorada. A ameaca ja se encontra bloqueada pelo Wazuh.
2026/06/06 15:15:01 active-response/bin/cisco_ips.py: Info: Acao repetida
    ignorada. A ameaca ja se encontra bloqueada pelo Wazuh.
2026/06/06 15:25:02 active-response/bin/cisco_ips.py: Custom IPS iniciado ->
    Comando: DELETE | Regra: 100302
2026/06/06 15:25:02 active-response/bin/cisco_ips.py: Timeout (Stateful). A
    remover protecao TCP Intercept.
2026/06/06 15:25:05 active-response/bin/cisco_ips.py: SUCESSO: Comandos
    aplicados no router (DELETE)!
```

Listing 10.13: Logs do *active response* no Wazuh server

A captura de pacotes realizada no Wireshark permite visualizar o impacto do ataque antes de a medida de segurança ser aplicada no router. Durante esta fase inicial, o atacante inunda a rede com pacotes TCP com a flag SYN ativada. Como o router atua apenas como dispositivo de encaminhamento nesta fase, estes pedidos chegam ao agente Xubuntu (vítima do ataque), que aloca recursos do sistema e tenta responder a cada pedido forjado com pacotes com as flags SYN e ACK ativadas, evidenciando o esgotamento de recursos no alvo (Figura 30).

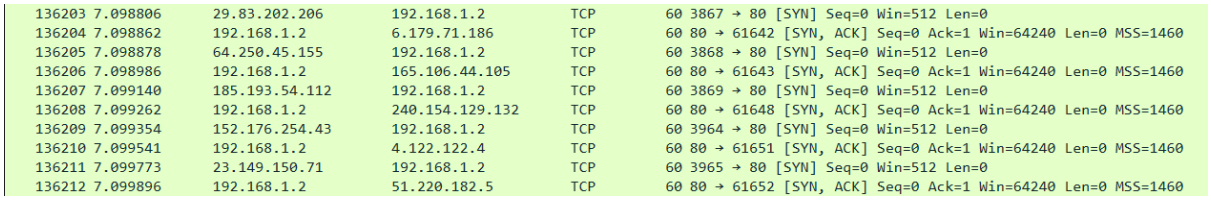


Figura 30: Pedidos de ligação fraudulentos e tentativas de resposta do router

Passados alguns momentos no meio destes pacotes TCP [22] é possivel ver o início de uma ligação Telnet [33] com alguns pacotes "espalhados"que tem dificuldade de iniciar com o router dado o ataque está a sofrer (erro no pacote Telnet *Previous segment not captured*) (Figura 31).

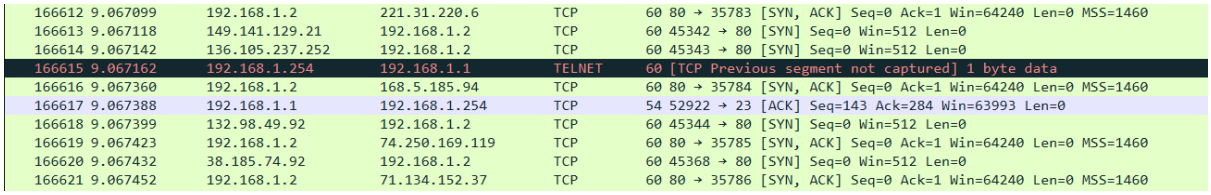
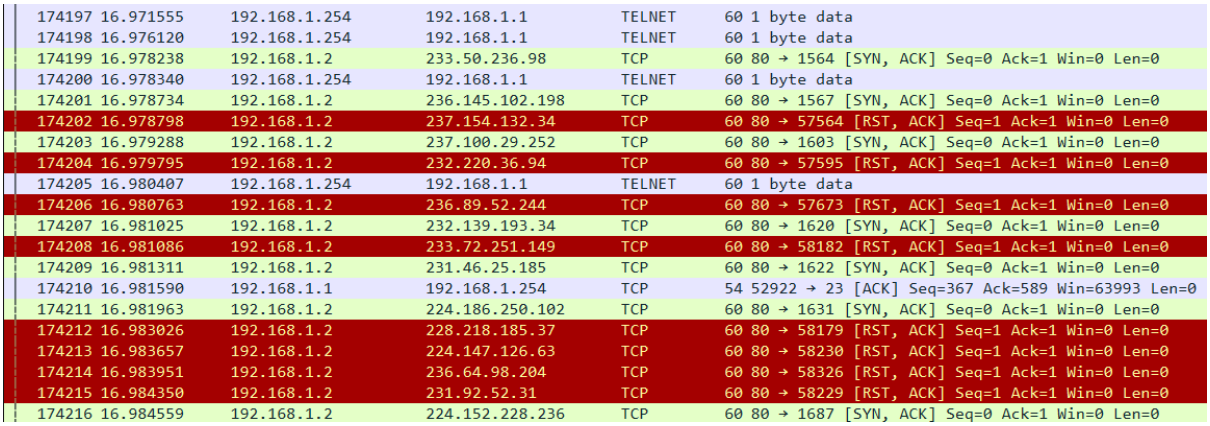


Figura 31: Início da ligação Telnet

Assim que a ligação Telnet consegue instruir o router para ativar a ferramenta *TCP Intercept* do Cisco IOS, a dinâmica da rede altera-se drasticamente. O router deixa de encaminhar os pacotes maliciosos para o agente Xubuntu e passa a intercetá-los. Como o volume do ataque ultrapassa o limiar de conexões incompletas configurado, o router entra em modo agressivo (*getting aggressive*). É possível observar no Wireshark que o próprio router começa a gerar e a enviar segmentos TCP com as flags RST e ACK ativadas em nome do IP do agente (192.168.1.2). Isto significa que o router está a descartar ativamente os pedidos fraudulentos e a redefinir as conexões para proteger a sua própria memória e a colocar-se entre o agente e o tráfego malicioso (Figura 32).



174197	16.971555	192.168.1.254	192.168.1.1	TELNET	60 1 byte data
174198	16.976120	192.168.1.254	192.168.1.1	TELNET	60 1 byte data
174199	16.978238	192.168.1.2	233.50.236.98	TCP	60 80 → 1564 [SYN, ACK] Seq=0 Ack=1 Win=0 Len=0
174200	16.978340	192.168.1.254	192.168.1.1	TELNET	60 1 byte data
174201	16.978734	192.168.1.2	236.145.102.198	TCP	60 80 → 1567 [SYN, ACK] Seq=0 Ack=1 Win=0 Len=0
174202	16.978798	192.168.1.2	237.154.132.34	TCP	60 80 → 57564 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
174203	16.979288	192.168.1.2	237.100.29.252	TCP	60 80 → 1603 [SYN, ACK] Seq=0 Ack=1 Win=0 Len=0
174204	16.979795	192.168.1.2	232.220.36.94	TCP	60 80 → 57595 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
174205	16.980407	192.168.1.254	192.168.1.1	TELNET	60 1 byte data
174206	16.980763	192.168.1.2	236.89.52.244	TCP	60 80 → 57673 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
174207	16.981025	192.168.1.2	232.139.193.34	TCP	60 80 → 1620 [SYN, ACK] Seq=0 Ack=1 Win=0 Len=0
174208	16.981086	192.168.1.2	233.72.251.149	TCP	60 80 → 58182 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
174209	16.981311	192.168.1.2	231.46.25.185	TCP	60 80 → 1622 [SYN, ACK] Seq=0 Ack=1 Win=0 Len=0
174210	16.981590	192.168.1.1	192.168.1.254	TCP	54 52922 → 23 [ACK] Seq=367 Ack=589 Win=63993 Len=0
174211	16.981963	192.168.1.2	224.186.250.102	TCP	60 80 → 1631 [SYN, ACK] Seq=0 Ack=1 Win=0 Len=0
174212	16.983026	192.168.1.2	228.218.185.37	TCP	60 80 → 58179 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
174213	16.983657	192.168.1.2	224.147.126.63	TCP	60 80 → 58230 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
174214	16.983951	192.168.1.2	236.64.98.204	TCP	60 80 → 58326 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
174215	16.984350	192.168.1.2	231.92.52.31	TCP	60 80 → 58229 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
174216	16.984559	192.168.1.2	224.152.228.236	TCP	60 80 → 1687 [SYN, ACK] Seq=0 Ack=1 Win=0 Len=0

Figura 32: Router a recusar pacotes maliciosos

No final da captura é possível observar uma nova ligação Telnet [33] (corresponde ao mecanismo statefull do Wazuh *active response*).

No cenário em que o ataque *SYN flood* é direcionado diretamente à interface do próprio router (IP 192.168.2.254), a principal diferença é o impacto na rede. Em vez de o tráfego ser encaminhado (*forwarded*) para a rede interna, este atinge diretamente o *Control Plane* do equipamento Cisco.

Isto representa um risco sistémico maior. Enquanto o ataque anterior compromete os recursos do agente Xubuntu, o esgotamento de CPU e memória do router (ao tentar gerir milhares de conexões falsas) pode causar uma falha da rede total (DoS), isolando as sub-redes.

Apesar da mudança de alvo, a resposta do IPS opera de forma idêntica. A regra da ACL dedicada ao router (`permit tcp any host 192.168.2.254 eq 80 log`) assegura a deteção atempada pelo Wazuh. A grande diferença prática é o propósito da mitigação: o *TCP Intercept* atua agora diretamente na auto-preservação do router.

11 Conclusão

Este trabalho permitiu estudar o Wazuh em contexto laboratorial, desde a sua componente teórica até à aplicação prática em cenários de deteção e resposta a incidentes. Ao longo do relatório foi possível compreender que a ferramenta não se limita apenas à recolha de logs, mas funciona como uma plataforma de monitorização e correlação de eventos, capaz de apoiar mecanismos IDS, HIDS, XDR e, quando combinada com respostas ativas, comportamentos próximos de um sistema IPS.

Inicialmente foi definida a proposta do projeto e a necessidade de criar um ambiente controlado para analisar ataques e respostas sem comprometer sistemas reais. A topologia em GNS3 permitiu separar uma rede protegida de uma rede desprotegida, criando condições adequadas para observar o comportamento do atacante, do agente Xubuntu, do servidor Wazuh e do router Cisco. Esta organização foi essencial para validar a comunicação entre dispositivos, testar políticas de segurança e perceber o impacto das medidas aplicadas em diferentes pontos da rede.

A análise da ferramenta Wazuh demonstrou a importância dos seus principais componentes. O Wazuh Manager foi identificado como o elemento responsável pela receção, descodificação e correlação dos eventos, enquanto o Filebeat assegura o encaminhamento dos alertas para o Wazuh Indexer. O Indexer armazena e disponibiliza os dados para pesquisa, e o Dashboard permite visualizar alertas, eventos e informação de gestão. Esta separação de funções ajudou a compreender melhor a arquitetura All-in-one usada no laboratório e a forma como os componentes cooperam para transformar eventos recolhidos em informação útil para análise de segurança.

Também foi analisada a comunicação entre componentes, incluindo o registo dos agentes, a troca de eventos entre agente e servidor, a utilização de portas específicas e a proteção baseada em mecanismos criptográficos. Esta parte permitiu perceber que a segurança do sistema não depende apenas das regras de deteção, mas também da forma como os componentes comunicam, se autenticam e protegem os dados transmitidos.

Na parte prática, foram aplicadas políticas de segurança orientadas a três cenários principais: bloqueio de um IP malicioso em serviços Web, bloqueio dinâmico contra ataques de força bruta em SSH e Telnet, e proteção contra *port scans* e *SYN floods*. No primeiro caso, a utilização de uma lista CDB permitiu identificar um IP malicioso e acionar a resposta ativa *firewall-drop*, bloqueando a origem do ataque após a primeira interação HTTP. Apesar de o primeiro pedido ainda ocorrer devido à natureza reativa do mecanismo, os pedidos seguintes foram bloqueados ao nível da rede.

No segundo cenário, a simulação de força bruta com Hydra demonstrou que as regras nativas do Wazuh conseguem correlacionar múltiplas falhas de autenticação e acionar uma resposta ativa. A análise dos logs e dos pacotes capturados mostrou que, após a ativação da regra, as comunicações do atacante passaram a falhar, confirmando a eficácia da política configurada no agente.

Nos testes contra *port scans* e *SYN floods*, a integração com o router Cisco foi o ponto mais relevante. Através de ACLs, Syslog, descodificadores personalizados, regras de correlação temporal e um script Python de resposta ativa, o Wazuh passou a interagir com a infraestrutura de rede. Esta abordagem permitiu aplicar rotas nulas para mitigar varrimentos de portos e ativar o *TCP Intercept* para reduzir o impacto de ataques SYN Flood. Assim, o sistema deixou de atuar apenas no agente e passou também a influenciar o comportamento do router, aproximando-se de uma resposta IPS distribuída.

A análise das experiências confirmou que o Wazuh é eficaz na deteção e correlação de eventos, mas também evidenciou limitações importantes. Muitas respostas dependem da existência de logs, o que significa que alguns ataques só são mitigados após uma primeira interação ou depois de ser atingido um determinado limiar de eventos. Ainda assim, com regras bem definidas, listas de reputação, descodificadores adequados e automação externa, a ferramenta consegue responder de forma rápida e útil em ambiente laboratorial.

Conclui-se que os objetivos definidos no início do trabalho foram alcançados. Foi possível instalar e configurar uma topologia funcional, compreender a arquitetura do Wazuh, analisar a comunicação entre os seus componentes, aplicar políticas de deteção e resposta, e validar o comportamento do sistema perante diferentes ataques. O trabalho mostrou ainda que o Wazuh pode ser uma solução bastante interessante para monitorização e defesa em ambientes controlados, especialmente quando complementado com respostas ativas e integração com equipamentos de rede.

Como trabalho futuro, seria interessante testar a mesma arquitetura num ambiente com maior número de agentes, integrar fontes adicionais de logs, melhorar os scripts de resposta ativa e comparar o desempenho entre arquiteturas All-in-one, Single-node e Multi-node. Também seria relevante explorar políticas mais granulares, dashboards personalizados e mecanismos de resposta com menor risco de falsos positivos, aproximando ainda mais o laboratório de um cenário real de produção.

Referências bibliográficas

[1] P. Mockapetris. (1987) Domain names - implementation and specification. (acedido em 06/06/2026). [Online]. Available: https://www.rfc-editor.org/rfc/rfc1035

[2] J. Postel and J. Reynolds. (1985) File transfer protocol. (acedido em 06/06/2026). [Online]. Available: https://www.rfc-editor.org/rfc/rfc959

[3] Wazuh. (2026) Wazuh - open source xdr. open source siem. (acedido em 16/05/2026). [Online]. Available: https://wazuh.com/

[4] TechSpot. (2026) Vmware workstation pro. (acedido em 14/05/2026). [Online]. Available: https://www.techspot.com/downloads/189-vmware-workstation-for-windows.html

[5] GNS3. (2026) Releases · gns3/gns3-gui. (acedido em 14/05/2026). [Online]. Available: https://github.com/GNS3/gns3-gui/releases

[6] Wireshark Foundation. (2026) Download wireshark. (acedido em 14/05/2026). [Online]. Available: https://www.wireshark.org/download.html

[7] Xubuntu. (2026) Download. (acedido em 14/05/2026). [Online]. Available: https://xubuntu.org/download/

[8] Wazuh. (2026) Virtual machine (amazon vm). (acedido em 15/05/2026). [Online]. Available: https://documentation.wazuh.com/current/deployment-options/virtual-machine/virtual-machine.html

[9] World Wide Web Consortium. (2008) Extensible markup language (xml) 1.0. (acedido em 07/06/2026). [Online]. Available: https://www.w3.org/TR/xml/

[10] Wazuh. (2026) User manual. (acedido em 24/04/2026). [Online]. Available: https://documentation.wazuh.com/current/user-manual/index.html

[11] Kali Linux. (2026) hydra | kali linux tools. (acedido em 06/06/2026). [Online]. Available: https://www.kali.org/tools/hydra/

[12] Kali Linux. (2026) nmap | kali linux tools. (acedido em 06/06/2026). [Online]. Available: https://www.kali.org/tools/nmap/

[13] Nmap Project. (2026) Nmap reference guide. (acedido em 06/06/2026). [Online]. Available: https://nmap.org/book/man.html

[14] Kali Linux. (2026) hping3 | kali linux tools. (acedido em 06/06/2026). [Online]. Available: https://www.kali.org/tools/hping3/

[15] Linux man-pages project. (2026) ping(8) - linux manual page. (acedido em 06/06/2026). [Online]. Available: https://man7.org/linux/man-pages/man8/ping.8.html

[16] Wazuh. (2026) Step-by-step installation - wazuh server. (acedido em 24/04/2026). [Online]. Available: https://documentation.wazuh.com/current/installation-guide/wazuh-server/step-by-step.html

[17] Wazuh. (2026) Wazuh indexer. (acedido em 24/04/2026). [Online]. Available: https://documentation.wazuh.com/current/user-manual/wazuh-indexer/index.html

[18] Wazuh. (2026) Creating custom dashboards. (acedido em 24/04/2026). [Online]. Available: https://documentation.wazuh.com/current/user-manual/wazuh-dashboard/creating-custom-dashboards.html

[19] Wazuh. (2026) Wazuh agent. (acedido em 24/04/2026). [Online]. Available: https://documentation.wazuh.com/current/installation-guide/wazuh-agent/index.html

[20] Wazuh. (2026) Architecture. (acedido em 24/04/2026). [Online]. Available: https://documentation.wazuh.com/current/getting-started/architecture.html

[21] Wazuh. (2026) Wazuh server cluster. (acedido em 04/06/2026). [Online]. Available: https://documentation.wazuh.com/current/user-manual/wazuh-server-cluster/index.html

[22] W. Eddy. (2022) Transmission control protocol (tcp). (acedido em 06/06/2026). [Online]. Available: https://www.rfc-editor.org/rfc/rfc9293

[23] Wazuh. (2026) Agent enrollment. (acedido em 04/06/2026). [Online]. Available: https://documentation.wazuh.com/current/user-manual/agent/agent-enrollment/index.html

[24] Wazuh. (2026) Required ports. (acedido em 04/06/2026). [Online]. Available: https://documentation.wazuh.com/current/getting-started/architecture.html#required-ports

[25] Wazuh. (2026) Wazuh manager identity verification. (acedido em 04/06/2026). [Online]. Available: https://documentation.wazuh.com/current/user-manual/agent/agent-enrollment/security-options/manager-identity-verification.html

[26] Wazuh. (2026) Wazuh agent identity verification. (acedido em 04/06/2026). [Online]. Available: https://documentation.wazuh.com/current/user-manual/agent/agent-enrollment/security-options/agent-identity-verification.html

[27] Wazuh. (2026) Benefits of using aes in our communications. (acedido em 24/04/2026). [Online]. Available: https://wazuh.com/blog/benefits-of-using-aes-in-our-communications/

[28] P. Srisuresh and K. Egevang. (2001) Traditional ip network address translator (traditional nat). (acedido em 06/06/2026). [Online]. Available: https://www.rfc-editor.org/rfc/rfc3022

[29] R. Droms. (1997) Dynamic host configuration protocol. (acedido em 06/06/2026). [Online]. Available: https://www.rfc-editor.org/rfc/rfc2131

[30] J. Postel. (1981) Internet protocol. (acedido em 06/06/2026). [Online]. Available: https://www.rfc-editor.org/rfc/rfc791

[31] T. Ylonen and C. Lonvick. (2006) The secure shell (ssh) transport layer protocol. (acedido em 06/06/2026). [Online]. Available: https://www.rfc-editor.org/rfc/rfc4253

[32] R. Fielding, M. Nottingham, and J. Reschke. (2022) Http semantics. (acedido em 06/06/2026). [Online]. Available: https://www.rfc-editor.org/rfc/rfc9110

[33] J. Postel and J. Reynolds. (1983) Telnet protocol specification. (acedido em 06/06/2026). [Online]. Available: https://www.rfc-editor.org/rfc/rfc854

[34] J. Postel. (1980) User datagram protocol. (acedido em 06/06/2026). [Online]. Available: https://www.rfc-editor.org/rfc/rfc768

[35] J. Postel. (1981) Internet control message protocol. (acedido em 06/06/2026). [Online]. Available: https://www.rfc-editor.org/rfc/rfc792

[36] QEMU. (2026) qemu-img. (acedido em 14/05/2026). [Online]. Available: https://www.qemu.org/docs/master/tools/qemu-img.html

[37] Wazuh. (2026) Virtual machine (vm). (acedido em 24/04/2026). [Online]. Available: https://documentation.wazuh.com/current/deployment-options/virtual-machine/virtual-machine.html

[38] C. Solutions. (2026) qemu-img for windows. (acedido em 15/05/2026). [Online]. Available: https://cloudbase.it/qemu-img-windows/

[39] E. Rescorla. (2000) Http over tls. (acedido em 06/06/2026). [Online]. Available: https://www.rfc-editor.org/rfc/rfc2818

[40] E. Rescorla. (2018) The transport layer security (tls) protocol version 1.3. (acedido em 06/06/2026). [Online]. Available: https://www.rfc-editor.org/rfc/rfc8446

Anexos

Esta secção reúne os anexos técnicos utilizados como apoio à configuração e replicação da topologia laboratorial. O objetivo dos anexos é concentrar os procedimentos mais operacionais, como comandos, parametrizações, criação de templates e validações, sem sobrecarregar o corpo principal do relatório. Assim, cada anexo funciona como um guia de consulta independente, permitindo compreender o propósito de cada etapa e repetir a configuração do ambiente de testes de forma organizada.

Os anexos encontram-se organizados pela ordem lógica de preparação da infraestrutura: primeiro são tratadas as imagens das máquinas virtuais, depois a integração no GNS3, a configuração da rede e, por fim, os serviços necessários para a comunicação, monitorização e realização das experiências.

- **Anexo A** – Conversão de Imagens para Formato QEMU;
- **Anexo B** – Preparação do Ambiente GNS3;
- **Anexo C** – Configuração do Router;
- **Anexo D** – Configuração do Servidor Wazuh;
- **Anexo E** – Configuração do Agente Wazuh;
- **Anexo F** – Configuração de Serviços SSH e Apache;
- **Anexo G** – Configuração do Telnet [33].
- **Anexo H** – Consulta de Comandos e Configurações
- **Anexo I** – Script de Python
- **Anexo J** – Ferramentas Kali

A Conversão de Imagens para Formato QEMU

Este anexo descreve a preparação inicial das imagens das máquinas virtuais utilizadas na topologia. A conversão para o formato QEMU/*qcow2* foi necessária para permitir a execução das máquinas diretamente no ambiente GNS3, evitando dependências adicionais de outros hipervisores e garantindo maior integração com a simulação de rede.

São apresentados os comandos usados para converter a imagem do Wazuh Server e a imagem do Xubuntu, bem como a movimentação dos ficheiros resultantes para a pasta de imagens do GNS3. Desta forma, o anexo documenta a etapa que torna as máquinas virtuais compatíveis com a infraestrutura laboratorial.

A.1 Procedimento de conversão

Para otimizar os recursos de hardware e evitar a execução de vários hipervisores em simultâneo, tirou-se partido do módulo de virtualização nativo do GNS3[5], baseado em QEMU[36]. Por este motivo, foi necessário converter alguns ficheiros descarregados para o formato ***qcow2***, permitindo a sua importação direta no GNS3.

A.1.1 Conversão Wazuh-Server

A máquina virtual do Wazuh[37] inclui o sistema operativo Amazon Linux 2023, o Server[16], o Indexer[17] e o Dashboard[18]. O ficheiro da VM no formato *Open Virtual Appliance* (*OVA*) foi reaproveitado para criar uma GNS3 Appliance.

Para realizar a conversão dos ficheiros para o formato QEMU[38] e preparar a sua utilização como appliance no GNS3, foi seguido um conjunto de comandos.

Os comandos necessários estão representados abaixo:

- Abrir o terminal e aceder ao local de instalação (**A.1**):

```
cd C:\Users\USERNAME\Downloads\qemu-img-win-x64-2_3_0-001\qemu-img-win-
    x64-2_3_0
```

Listing A.1: Acesso à pasta do qemu-img

- Executar o seguinte comando no terminal (**A.2**):

```
.\qemu-img.exe convert -p -f vmdk -O qcow2 wazuh-4.14.5-disk.vmdk wazuh-
    qemu.qcow2
```

Listing A.2: Conversão da imagem Wazuh para qcow2

- Mover o novo ficheiro para a pasta das imagens QEMU (**A.3**):

```
mv wazuh-qemu.qcow2 C:\Users\orlan\GNS3\images\QEMU
```

Listing A.3: Movimentação da imagem Wazuh para a pasta QEMU

A.1.2 Conversão Xubuntu

- Abrir o terminal e aceder ao local de instalação:

```
cd C:\Users\USERNAME\Downloads\qemu-img-win-x64-2_3_0-001\qemu-img-win-
    x64-2_3_0
```

Listing A.4: Acesso à pasta do qemu-img

- Executar o seguinte comando no terminal:

```
.\qemu-img.exe convert -p -f vdi -O qcow2 wazuh-agent.vdi xubuntu-agent.
    qcow2
```

Listing A.5: Conversão da imagem Xubuntu para qcow2

- Ainda com o terminal aberto, mover o ficheiro para a pasta das imagens QEMU:

```
mv xubuntu-agent.qcow2 C:\Users\orlan\GNS3\images\QEMU
```

Listing A.6: Movimentação da imagem Xubuntu para a pasta QEMU

B Preparação do Ambiente GNS3

Este anexo explica como as imagens previamente convertidas foram associadas ao GNS3 através da criação de templates QEMU. Esta etapa é essencial para que o Wazuh Server e o agente Xubuntu possam ser adicionados à topologia como dispositivos reutilizáveis, com recursos de memória, disco e consola devidamente definidos.

Além da criação dos templates, o anexo mostra as opções selecionadas no GNS3 e as configurações mínimas necessárias para garantir que as máquinas arrancam corretamente e ficam disponíveis para ligação aos restantes elementos da rede.

B.1 Associação das máquinas virtuais ao GNS3

Após os procedimentos realizados na secção **A.1**, as respetivas imagens das máquinas virtuais devem ser associadas ao GNS3. Este processo inclui a criação do template da appliance, a seleção da imagem QEMU e a definição dos recursos necessários para a sua execução.

É importante garantir que todas as máquinas virtuais estão associadas ao GNS3, para que a topologia fique funcional e os dispositivos consigam comunicar entre si.

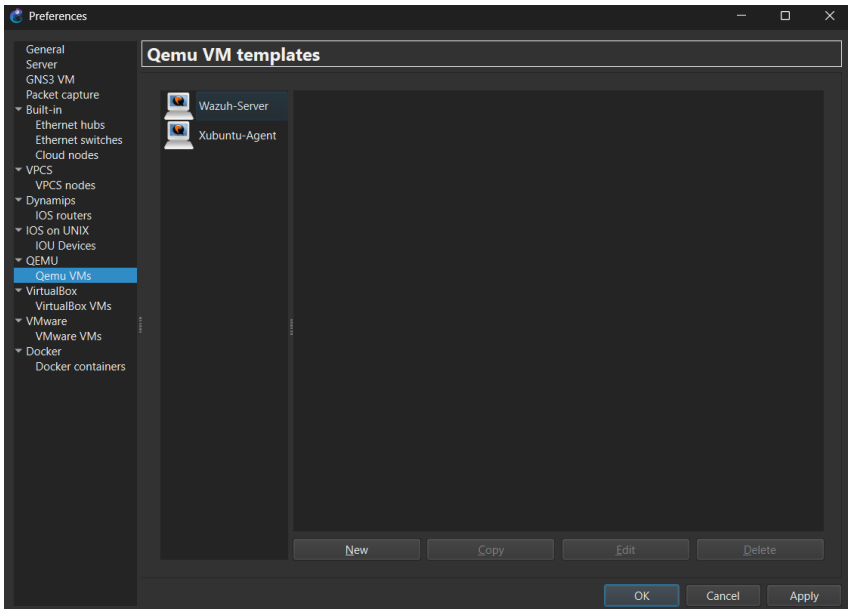


Figura 33: Templates QEMU disponíveis no GNS3

B.1.1 Criação e Associação Template Wazuh VM

Para criar a appliance do Wazuh Server no GNS3, devem ser seguidos alguns passos como o upload da imagem ao servidor GNS3, a adição do template e a configuração dos mesmos, para que os virtualizadores cumpram com os requisitos mínimos.

Upload da Imagem QUEMU do WAZUH no GNS3:

- Aceder a *Edit* > *Preferences* > *QEMU VMs* > *New* e selecionar a opção para executar a máquina virtual na GNS3 VM, conforme apresentado na **Figura 34**.

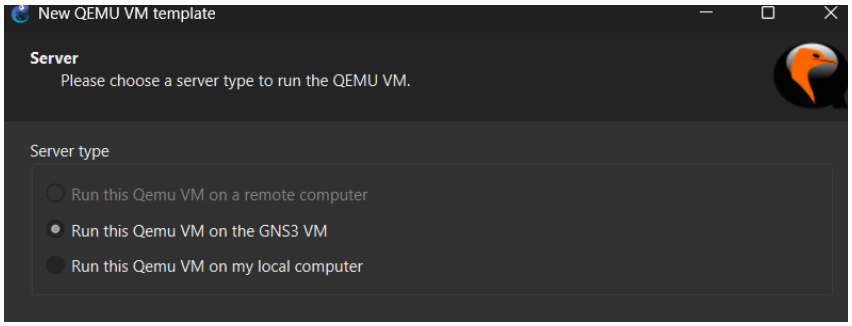


Figura 34: Criação de um novo template QEMU

- Definir o nome da máquina virtual como *Wazuh-Server*.
- Selecionar a opção *New Image* e indicar o ficheiro *wazuh-qemu.qcow2*, conforme apresentado na **Figura 35**.

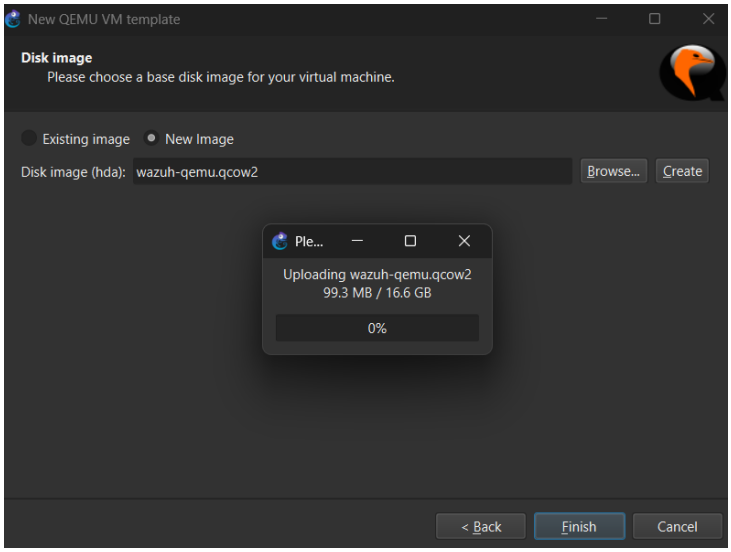


Figura 35: Seleção da imagem QEMU para o template

Depois de iniciar o carregamento da imagem, é necessário aguardar até que o upload seja concluído antes de avançar para a configuração dos recursos da máquina virtual.

Criação do Template apartir da imagem carregada

Após o carregamento da imagem,deve-se criar o template.

- *Edit* > *Preferences* > *QEMU VMs* > *New* e selecionar a opção para executar a máquina virtual na GNS3 VM 34
- Definir o nome da máquina virtual como *Wazuh*.
- Atribuir *8192 MB* de RAM, conforme apresentado na **Figura 37**.
- Definir o tipo de consola como *telnet*, conforme apresentado na **Figura 38**
- Selecionar Imagem: selecione a imagem wazuh-quemu.qcow2 conforme apresentado na (**Figura 36**)

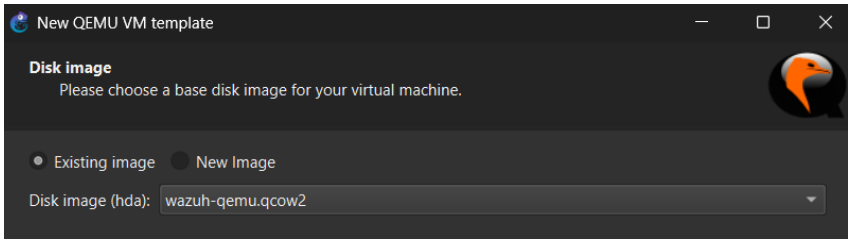


Figura 36: Add Template

Definições da máquina virtual Wazuh

Após a criação do template, devem ser ajustadas as definições principais da máquina virtual para garantir o seu funcionamento correto no GNS3.

- Definir o nome da máquina virtual como *Wazuh*.
- Atribuir *8192 MB* de RAM, conforme apresentado na **Figura 37**.

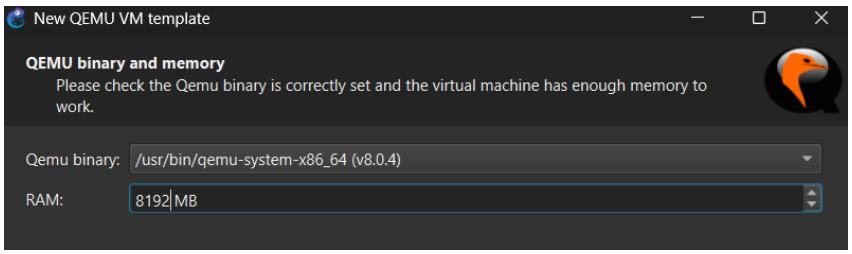


Figura 37: Definição da memória RAM da máquina virtual

- Definir o tipo de consola como *telnet*, conforme apresentado na **Figura 38**.

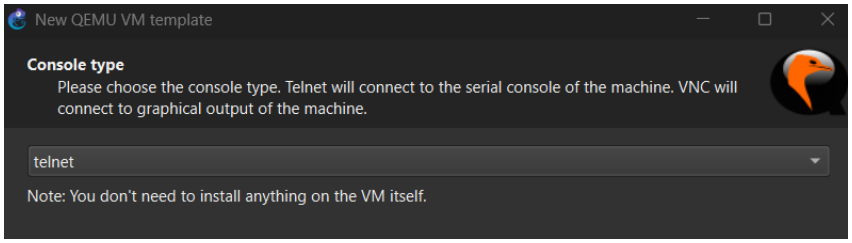


Figura 38: Definição do tipo de consola como Telnet

- Em *Edit* > *Preferences* > *QEMU VMs* > *Wazuh-Server* > *Edit*, definir *4 vCPUs* (*General Settings*), a interface de disco como *SCSI* (Figura 39) e a interface de rede (*Network* > *Type*) como *Paravirtualized Network I/O* (*virtio-net-pci*).

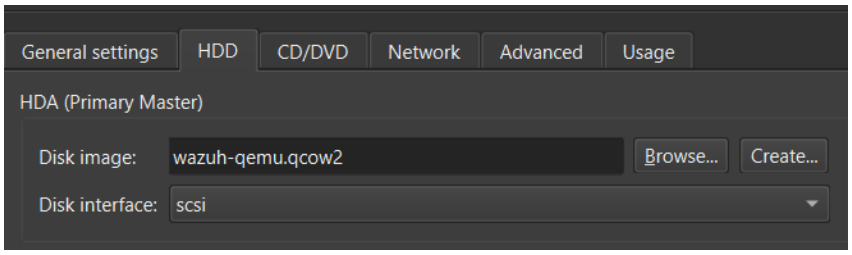


Figura 39: Wazuh hdd formato

- Em Advanced > Options adicione -cpu host"e clique em "OK"

B.1.2 Criação e associação template Xubuntu VM

A associação da máquina virtual Xubuntu é semelhante ao procedimento descrito na secção B.1.1. As diferenças encontram-se apenas nos recursos atribuídos e na imagem selecionada. Neste caso, deve ser utilizada a imagem *xubuntu-agent.qcow2* em vez da imagem *wazuh-qemu.qcow2*.

Definições da máquina virtual Xubuntu

- Definir o nome da máquina virtual como *Xubuntu-Agent*.
- Atribuir *2048 MB* de RAM, conforme apresentado na **Figura 37**.
- Definir o tipo de consola como *telnet*, conforme apresentado na **Figura 38**.
- Em *Edit* > *Preferences* > *QEMU VMs* > *Xubuntu-Agent* > *Edit*, definir *2 vCPUs* (*General Settings*), a interface de disco como *SCSI* e a interface de rede (*Network* > *Type*) como *Paravirtualized Network I/O* (*virtio-net-pci*).

C Configuração do Router

Este anexo apresenta a configuração do router responsável pela interligação dos segmentos da topologia. São documentadas as definições das interfaces, o endereçamento IP [30], o NAT [28] e as linhas de acesso, uma vez que estes elementos garantem a conectividade entre as máquinas e o acesso aos serviços necessários.

A configuração aqui descrita permite compreender como foi estruturada a comunicação entre o servidor Wazuh, o agente, o atacante e as redes externas simuladas, servindo também como referência para validar problemas de conectividade durante os testes.

C.1 Configuração do Router

Para tornar possível a execução dos testes, o router deve estar previamente configurado. Para isso, devem ser seguidas as orientações apresentadas abaixo:

A memória RAM do router foi aumentada para 1024 MB.

- Entrar no modo de configuração global do router:

```
enable
configure terminal
```

Listing C.1: Acesso ao modo de configuração do router

- Configurar o domínio e criar o utilizador local para acesso remoto:

```
ip domain name lab.local
username cisco privilege 15 secret cisco
```

Listing C.2: Configuração de domínio e utilizador

- Aplicar as configurações básicas e desativar serviços não essenciais:

```
hostname R1
no ip bootp server
no ip domain lookup
no cdp run
no ip http server
no ip http secure-server
```

Listing C.3: Configuração básica do router

- Configurar os endereços IP [30], NAT [28] e reassemblagem nas interfaces Ethernet:

```
interface Ethernet0/0
 ip address 192.168.1.254 255.255.255.0
 ip nat inside
 ip virtual-reassembly in
 no shut
exit
interface Ethernet0/1
 ip address dhcp
 ip nat outside
 ip virtual-reassembly in
 no shut
exit
interface Ethernet0/2
 ip address 192.168.2.254 255.255.255.0
 ip nat inside
 ip virtual-reassembly in
 no shut
exit
```

Listing C.4: Configuração das interfaces Ethernet e NAT

- Criar as Listas de Acesso para o NAT e configurar o NAT Overload [28]:

```
access-list 1 permit 192.168.1.0 0.0.0.255
access-list 2 permit 192.168.2.0 0.0.0.255
ip nat inside source list 1 interface Ethernet0/1 overload
ip nat inside source list 2 interface Ethernet0/1 overload
```

Listing C.5: Configuração do NAT Overload

- Configurar as linhas de acesso (Consola, Auxiliar e VTY):

```
line con 0
 exec-timeout 0 0
 privilege level 15
 logging synchronous
exit
line aux 0
 exec-timeout 0 0
 privilege level 15
 logging synchronous
exit
line vty 0 4
 login local
 transport input all
exit
```

Listing C.6: Configuração das linhas de acesso do router

- Configurar o envio de logs e a sua estrutura (Syslog):

```
no service timestamps log datetime msec
no service timestamps debug datetime msec
service timestamps log uptime
configure terminal
logging on
logging host 192.168.1.1 transport udp port 514
logging trap informational
```

Listing C.7: Configuração do servidor de Syslog

- Sair do modo de configuração e gravar as alterações:

```
end
write memory
```

Listing C.8: Gravação da configuração do router

D Configuração do Servidor Wazuh

Este anexo detalha a preparação do servidor Wazuh após a sua integração na topologia. A configuração inclui a atribuição de endereço IP estático [30], a definição da gateway e do DNS [1], a aplicação das alterações de rede e a atualização do sistema.

Estas etapas são importantes porque o servidor atua como componente central da solução, recebendo eventos dos agentes, disponibilizando o dashboard e permitindo a gestão das regras e respostas ativas usadas nas experiências.

D.1 Preparação da rede

Antes de configurar o endereço IP estático [30], é necessário desativar a configuração automática de rede do *cloud-init* e garantir que o serviço de rede está ativo.

```
sudo echo "network: {config: disabled}" | sudo tee /etc/cloud/cloud.cfg.d/99-
    disable-network-config.cfg
systemctl status network
sudo systemctl enable network
sudo systemctl restart network
sudo ifup eth0
```

Listing D.1: Preparação da configuração de rede no servidor

D.2 Configuração de IP estático

De seguida, deve ser aberto o ficheiro de configuração da interface de rede *eth0*.

```
sudo nano /etc/sysconfig/network-scripts/ifcfg-eth0
```

Listing D.2: Abertura do ficheiro de configuração da interface eth0

O conteúdo do ficheiro deve ser substituído pela configuração apresentada abaixo.

```
DEVICE=eth0
BOOTPROTO=none
ONBOOT=yes
TYPE=Ethernet
USERCTL=yes
IPADDR=192.168.1.1
NETMASK=255.255.255.0
GATEWAY=192.168.1.254
DNS1=8.8.8.8
```

Listing D.3: Configuração de IP estático na interface eth0

D.3 Instalação de Dependências e Alterações no sistema

Após guardar o ficheiro, as configurações devem ser aplicadas reiniciando o serviço de rede e confirmando o endereço atribuído à interface.

```
sudo systemctl restart network
ip a
sudo systemctl stop systemd-networkd.socket
sudo systemctl stop systemd-networkd.service
sudo systemctl disable systemd-networkd
sudo systemctl restart network
sudo reboot
```

Listing D.4: Aplicação e verificação da configuração de rede

Após o reinício do servidor, devem ser instaladas as dependências necessárias e atualizado o sistema.

```
sudo -i
sudo yum install python3-pip -y
sudo pip3 install netmiko
sudo pip3 install "ruamel.yaml<=0.17.21"
sudo yum install -y curl
sudo yum update
sudo reboot
```

Listing D.5: Instalação do curl e atualização do servidor

D.4 Validação no dashboard

Para aceder ao dashboard, deve ser inserido no navegador do Webterm o endereço HTTPS [39] do servidor Wazuh, protegido por TLS [40] (**D.6**):

```
https://ip_servidor
```

Listing D.6: Acesso ao dashboard do Wazuh

Após o acesso, é possível confirmar se o agente foi registado e se está a comunicar corretamente com o servidor, conforme apresentado nas figuras 40 e 41.

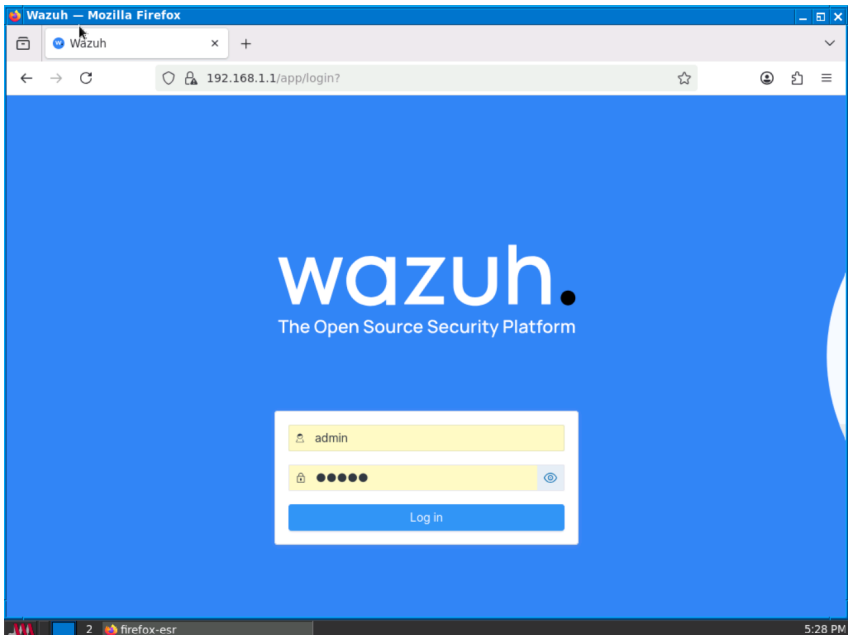


Figura 40: Página inicial do Wazuh

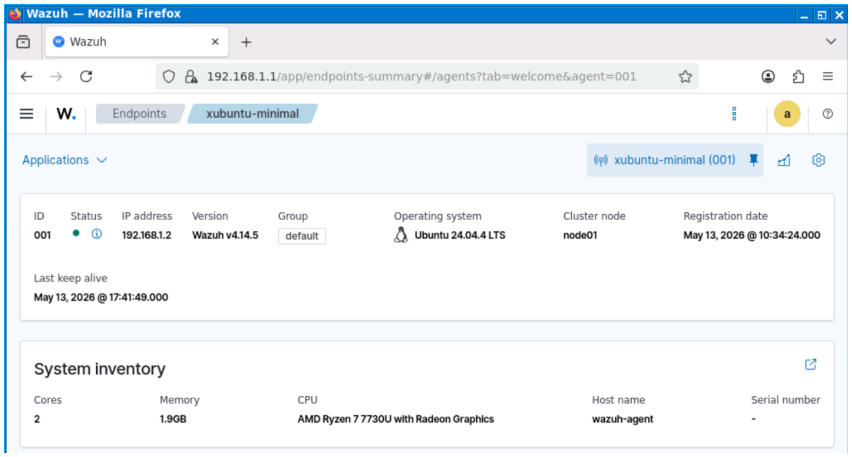


Figura 41: Análise do agente na dashboard

D.4.1 Configurar o Wazuh Server para receber Syslog:

```
sudo nano /var/ossec/etc/ossec.conf

<ossec_config>
  <remote>
    <connection>syslog</connection>
    <port>514</port>
    <protocol>udp</protocol>
    <allowed-ips>192.168.1.254</allowed-ips>
  </remote>
</ossec_config>
```

Listing D.7: Receção de syslogs no Wazuh server

E Configuração do Agente Wazuh

Este anexo descreve a instalação e associação do agente Wazuh na máquina Xubuntu. O objetivo é garantir que o cliente passa a enviar eventos de segurança para o Wazuh Server, permitindo a monitorização centralizada e a validação das experiências realizadas.

São apresentados os comandos de instalação, a edição do ficheiro de configuração do agente e o reinício do serviço, permitindo confirmar que o agente fica corretamente registado e operacional no dashboard.

E.1 Instalação e associação do agente Wazuh

O agente Wazuh[19] deve ser instalado na máquina Xubuntu e associado ao Wazuh Server através do ficheiro de configuração do agente. Este procedimento permite que o servidor receba os eventos do cliente e apresente o estado do agente no dashboard.

E.1.1 Instalação do agente

Para instalar o agente Wazuh, deve ser descarregado o pacote correspondente e ativado o serviço:

```
wget https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4
    .14.5-1_amd64.deb
sudo dpkg -i wazuh-agent_4.14.5-1_amd64.deb
sudo systemctl daemon-reload
sudo systemctl enable wazuh-agent
```

Listing E.1: Instalação do Wazuh Agent

E.1.2 Associação do endereço do servidor Wazuh

De seguida, deve ser editado o ficheiro de configuração do agente (**E.2**):

```
sudo nano /var/ossec/etc/ossec.conf
```

Listing E.2: Edição do ficheiro ossec.conf

No bloco <*server*>, deve ser definido o endereço IP do Wazuh Server(**Figura42**).

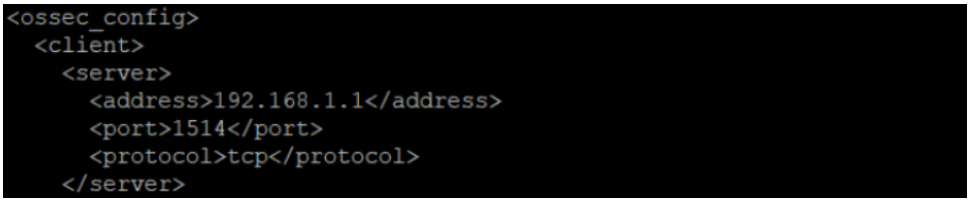


Figura 42: Configuração do ficheiro /var/ossec/etc/ossec.conf

Depois de guardar a configuração, o serviço do agente deve ser iniciado e reiniciado para aplicar as alterações. (**E.3**).

```
sudo systemctl start wazuh-agent
sudo systemctl restart wazuh-agent
```

Listing E.3: Arranque e reinício do Wazuh Agent

Configurar logs no servidor (**E.4**):

```
sudo nano /var/ossec/etc/ossec.conf
<logall>yes</logall>
sudo systemctl restart wazuh-manager
```

Listing E.4: Edição do ficheiro ossec.conf

F Configuração de Serviços SSH e Apache

Este anexo reúne a configuração dos serviços usados para gerar tráfego e eventos relevantes durante os testes. O SSH permite testar autenticações e tentativas de acesso remoto [31], enquanto o Apache disponibiliza um serviço Web que pode ser monitorizado pelo agente Wazuh.

A ativação destes serviços é necessária para criar cenários práticos de deteção e resposta, nomeadamente tentativas de ligação, recolha de logs e validação das regras aplicadas no servidor Wazuh.

F.1 Configuração do SSH no agente

O serviço SSH foi ativado no agente Xubuntu [31] para permitir ligações remotas e para criar uma fonte de eventos de autenticação. Estes eventos são úteis para testar a recolha de logs pelo Wazuh e para validar cenários relacionados com tentativas de acesso ao sistema.

Nesta etapa é feita a atualização dos repositórios, a instalação do servidor OpenSSH, a ativação automática do serviço no arranque e a confirmação do seu estado.

```
sudo apt update
sudo apt install openssh-server -y
sudo systemctl enable --now ssh
sudo systemctl status ssh
```

Listing F.1: Instalação e ativação do SSH no agente

F.2 Configuração do servidor Apache no agente

O Apache foi instalado no agente para disponibilizar um serviço Web simples dentro da topologia. Este serviço permite gerar pedidos HTTP a partir da máquina atacante [32] e, consequentemente, criar registos de acesso que podem ser monitorizados pelo agente Wazuh.

A configuração inclui a instalação do pacote, a ativação do serviço e a verificação do estado, garantindo que o servidor Web fica disponível para os testes.

```
sudo apt update
sudo apt install apache2 -y
sudo systemctl enable --now apache2
sudo systemctl status apache2
```

Listing F.2: Instalação do servidor Apache no agente

F.3 Configuração do SSH no servidor Wazuh

O SSH também foi configurado no servidor Wazuh [31] para permitir administração remota da máquina e facilitar a execução de comandos durante a preparação do ambiente.

Depois da instalação, o serviço é ativado, validado e o servidor é reiniciado para garantir que todas as alterações ficam aplicadas corretamente.

```
sudo dnf update -y
sudo dnf install openssh-server -y
sudo systemctl enable --now sshd
sudo systemctl status sshd
sudo reboot
```

Listing F.3: Instalação e ativação do SSH no servidor Wazuh

G Configuração do Telnet

Este anexo apresenta a configuração do Telnet no servidor e no agente Wazuh [33]. Embora seja um protocolo inseguro e desaconselhado em ambientes reais, foi incluído no laboratório para demonstrar um serviço vulnerável e facilmente observável durante experiências de deteção.

G.1 Configuração do Telnet no Servidor Wazuh

A instalação do cliente Telnet no servidor Wazuh permite testar ligações a serviços remotos [33] e validar a conectividade dentro da topologia. Apesar de não ser recomendado para utilização segura em produção, o Telnet é útil em ambiente laboratorial por permitir observar tráfego em texto claro e simplificar testes de comunicação.

```
sudo yum install telnet
```

Listing G.1: Instalação do Telnet no servidor Wazuh

G.2 Configuração do Telnet no Agente Wazuh

No agente Wazuh foi configurado o serviço Telnet para criar um alvo de acesso remoto vulnerável [33]. Esta configuração permite simular tentativas de ligação e autenticação, produzindo eventos que podem ser analisados pelo Wazuh no contexto das experiências de deteção e resposta.

Antes da instalação, é definido um servidor DNS para garantir resolução de nomes [1] durante a atualização dos pacotes. De seguida, o serviço é instalado, ativado através do *inetd* e reiniciado para aplicar a configuração.

```
echo "nameserver 8.8.8.8" | sudo tee /etc/resolv.conf
sudo apt update
sudo apt install telnetd
sudo systemctl status inetutils-inetd
sudo systemctl enable --now inetutils-inetd
echo "telnet stream tcp nowait root /usr/sbin/tcpd /usr/sbin/telnetd" | sudo
    tee -a /etc/inetd.conf
sudo systemctl restart inetutils-inetd
```

Listing G.2: Configuração do Telnet no cliente Wazuh

H Comandos de Configuração de consulta

Este anexo representa a consulta de ficheiros necessários para realização de configurações no Wazuh, bem como como proceder a leitura e configuração de logs.

H.1 Comandos de consulta e Configuração

H.1.1 Configuração de logs

Comandos para consultar logs recebidos no cliente (agente Wazuh) (**H.1**):

```
sudo cat /var/ossec/logs/ossec.log
sudo tail -f /var/ossec/logs/ossec.log
sudo tail -f /var/ossec/logs/active-responses.log
```

Listing H.1: Consulta de logs no agente Wazuh

Comandos para consultar logs recebidos no servidor (Wazuh manager) (**H.2**):

```
sudo tail -f /var/ossec/logs/active-responses.log
sudo cat /var/ossec/logs/archives/archives.log
sudo tail -f /var/ossec/logs/archives/archives.log
sudo cat /var/ossec/logs/archives/archives.log | grep 192.168.1.4
```

Listing H.2: Consulta de logs no Wazuh Manager

I Ficheiro Python Automatização de IPS

I.1 Script Python

```
#!/usr/bin/python3
import sys
import json
import re
from datetime import datetime
from netmiko import ConnectHandler

LOG_FILE = "/var/ossec/logs/active-responses.log"

def log_message(message):
    timestamp = datetime.now().strftime('\%Y/\%m/\%d \%H:\%M:\%S')
    with open(LOG_FILE, "a") as f:
        f.write(f"{timestamp} active-response/bin/cisco_ips.py: {message}\n")

def main():
    # Ler a primeira linha do STDIN (log)
    try:
        input_str = sys.stdin.readline().strip()
        if not input_str:
            return
        alert_data = json.loads(input_str)
    except Exception as e:
        log_message(f"Erro: Nao foi possivel processar o JSON inicial: {str(e)
    }")
        return

    # Extrair acao do Wazuh (add/delete)
    wazuh_command = alert_data.get("command", "")

    # Extrair a informacao necessaria do log
    parameters = alert_data.get("parameters", {})
    alert = parameters.get("alert", {})
```

```
rule_id = str(alert.get("rule", {}).get("id", ""))
full_log = alert.get("full_log", "")

if not wazuh_command or not rule_id:
    log_message("Erro: Comando ou ID da regra nao encontrados no log.")
    return

# HANDSHAKE STATEFUL (Documentacao Wazuh)
if wazuh_command == "add":
    action_key = "tcp_intercept"
    if rule_id == "100301":
        match = re.search(r'denied \w+ ([0-9\.]+)', full_log)
        if match:
            action_key = match.group(1) # Usa o IP do atacante como chave

    # Enviar check_keys para o STDOUT
    keys_msg = json.dumps({
        "version": 1,
        "origin": {"name": sys.argv[0], "module": "active-response"},
        "command": "check_keys",
        "parameters": {"keys": [action_key]}
    })
    print(keys_msg)
    sys.stdout.flush()

    # Ler imediatamente a resposta do Wazuh Manager (continue = ok)
    try:
        response_str = sys.stdin.readline().strip()
        response_data = json.loads(response_str)
        if response_data.get("command") != "continue":
            log_message("Info: Acao repetida ignorada. A ameaca ja se
encontra bloqueada pelo Wazuh.")
            return
    except Exception as e:
        log_message(f"Erro ao ler confirmacao do Wazuh: {str(e)}")
        return

log_message(f"Custom IPS iniciado -> Comando: {wazuh_command.upper()} |
Regra: {rule_id}")

# ligacao telnet (ssh nao e compativel entre wazuh server e imagem ios
antiga)
cisco_router = {
```

```
    'device_type': 'cisco_ios_telnet',
    'host': '192.168.1.254',
    'username': 'cisco',
    'password': 'cisco',
    'secret': 'cisco'
}

commands_to_send = []

# LOGICA DO NMAP SCAN (NULL ROUTE)
if rule_id == "100301":
    match = re.search(r'denied \w+ ([0-9\.]+)', full_log)
    if not match:
        log_message("Erro: logica do nmap falhou (IP do atacante nao
encontrado).")
        return
    malicious_ip = match.group(1)

    if wazuh_command == "add":
        commands_to_send = [f"ip route {malicious_ip} 255.255.255.255
Null0"]
        log_message(f"A injetar null route no router. IP alvo: {
malicious_ip}")
    elif wazuh_command == "delete":
        commands_to_send = [f"no ip route {malicious_ip} 255.255.255.255
Null0"]
        log_message(f"Timeout (Stateful). A remover null route para: {
malicious_ip}")

# LOGICA DO SYN FLOOD (TCP INTERCEPT)
elif rule_id == "100302":
    if wazuh_command == "add":
        commands_to_send = [
            "ip access-list extended SYN_PROTECT",
            "permit tcp any any eq 80",
            "exit",
            "ip tcp intercept list SYN_PROTECT",
            "ip tcp intercept mode intercept",
            "ip tcp intercept max-incomplete low 50 high 100",
            "ip tcp intercept connection-timeout 30",
            "ip tcp intercept drop-mode random"
        ]
```

```
            log_message("A injetar protecao TCP Intercept no router (anti SYN
    Flood).")
        elif wazuh_command == "delete":
            commands_to_send = [
                "no ip tcp intercept list SYN_PROTECT",
                "no ip access-list extended SYN_PROTECT"
            ]
            log_message("Timeout (Stateful). A remover protecao TCP Intercept
    .")

    else:
        log_message(f"Erro: Regra {rule_id} nao mapeada no script.")
        return

    # APLICAR COMANDOS NO ROUTER
    try:
        net_connect = ConnectHandler(**cisco_router)
        net_connect.enable()
        net_connect.send_config_set(commands_to_send)
        net_connect.disconnect()
        log_message(f"SUCESSO: Comandos aplicados no router ({wazuh_command.
    upper()})!")
    except Exception as e:
        log_message(f"Erro de ligacao ao router: {str(e)}")

if __name__ == "__main__":
    main()
```

Listing I.1: cisco_ips.py

J Ferramentas Kali

J.1 Ferramentas utilizadas no Kali Linux

Este anexo apresenta as ferramentas utilizadas no atacante Kali Linux durante as experiências. Para cada ferramenta é indicada a sua finalidade, uma forma simples de instalação e a sintaxe base usada nos testes, recorrendo às páginas oficiais do Hydra [11], hping3 [14], Nmap [12], ao guia de referência do Nmap [13], à documentação do ping [15] e à documentação técnica do ICMP [35].

J.1.1 Hydra

O *Hydra* [11] é uma ferramenta de testes de autenticação usada para validar a resistência de serviços como SSH [31], Telnet [33], FTP [2] ou HTTP [32] contra ataques de força bruta. Neste trabalho foi utilizada para gerar várias tentativas de autenticação SSH contra o agente Xubuntu, permitindo testar a deteção do Wazuh e a ativação da resposta ativa.

```
sudo apt update
sudo apt install hydra
```

Listing J.1: Instalação do Hydra

A sintaxe geral do Hydra segue a estrutura *utilizador*, conjunto de passwords e serviço de destino [11]:

```
hydra -l <utilizador> -P <ficheiro_passwords> <servico>://<ip_destino>
```

Listing J.2: Sintaxe geral do Hydra

No cenário testado foi usada a geração automática de passwords com um carácter, suficiente para provocar múltiplas falhas de autenticação:

```
hydra -l wazuh -x 1:1:abcdefghij ssh://192.168.1.2
```

Listing J.3: Ataque SSH com Hydra

J.1.2 Hping3

O *hping3* [14] é uma ferramenta de geração e manipulação de pacotes TCP [22]/IP [30]. Permite construir pacotes com flags específicas, alterar portas, usar endereços de origem aleatórios e gerar tráfego em grande volume. Neste trabalho foi usado para simular um ataque *SYN flood* contra o agente, atravessando o router Cisco.

```
sudo apt update
sudo apt install hping3
```

Listing J.4: Instalação do hping3

A sintaxe base usada para enviar segmentos TCP SYN para uma porta específica é:

```
hping3 -S -p <porta> <ip_destino>
```

Listing J.5: Sintaxe geral do hping3

No teste de *SYN flood*, foram usadas as opções `-S` para ativar a flag SYN, `-flood` para envio contínuo, `-rand-source` para usar endereços de origem aleatórios e `-p 80` para apontar ao serviço HTTP [32]:

```
hping3 -S --flood --rand-source -p 80 192.168.1.2
```

Listing J.6: Comando hping3 para SYN flood

J.1.3 Nmap

O *Nmap* é uma ferramenta de descoberta e auditoria de rede. Permite identificar hosts ativos, portas abertas, serviços expostos e, em alguns casos, versões de serviços. Neste trabalho foi usado para gerar tráfego de varrimento contra o router e validar a política de deteção de *port scans*.

```
sudo apt update
sudo apt install nmap
```

Listing J.7: Instalação do Nmap

A sintaxe mais simples do Nmap consiste em indicar o endereço IP de destino [30]:

```
nmap <ip_destino>
```

Listing J.8: Sintaxe geral do Nmap

No cenário analisado foi efetuado um varrimento ao router:

```
nmap 192.168.1.254
```

Listing J.9: Varrimento de portos com Nmap

J.1.4 Ping

O *ping* [15] utiliza ICMP [35] para testar conectividade IP [30] entre duas máquinas. Foi usado após a ativação das respostas ativas para confirmar se o atacante continuava a conseguir comunicar com o alvo.

```
sudo apt update
sudo apt install iputils-ping
```

Listing J.10: Instalação do ping

A sintaxe base consiste em indicar o endereço IP [30] ou nome DNS [1] de destino:

```
ping <ip_destino>
```

Listing J.11: Sintaxe geral do ping

Exemplos de utilização:

```
ping 192.168.1.2
ping 8.8.8.8
```

Listing J.12: Testes de conectividade com ping

Instituto Superior
de Engenharia
Politécnico de Coimbra